

Implementing Cumulative Functions with Generalized Cumulative Constraints

PIERRE SCHAUS*, UCLouvain, ICTEAM, Belgium

CHARLES THOMAS, UCLouvain, ICTEAM, Belgium

ROGER KAMEUGNE, UCLouvain, ICTEAM and University of Maroua, Belgium / Cameroon

Modeling scheduling problems with conditional time intervals and cumulative functions has become a common approach when using modern commercial constraint programming solvers. This paradigm enables the modeling of a wide range of scheduling problems, including those involving producers and consumers. However, it is unavailable in existing open-source solvers and practical implementation details remain undocumented. In this work, we present an implementation of this modeling approach using a single, generic global constraint called the Generalized Cumulative. We also introduce a novel timetabling filtering algorithm specifically designed to handle tasks defined on conditional time-intervals. Experimental results demonstrate that this approach, combined with the new filtering algorithm, performs competitively with existing solvers enabling the modeling of producer and consumer scheduling problems and effectively scales to large-scale problems.

1 Introduction

Constraint Programming (CP) is a declarative paradigm for solving combinatorial satisfaction and optimisation problems. A CP model consists of a set of *decision variables*, each associated with a finite domain of possible values, and a collection of *constraints* that restrict the combinations of values that the variables may simultaneously take. Solving alternates between *constraint propagation* — removing domain values that cannot appear in any solution — and *search*, typically a form of systematic backtracking. A key strength of CP is the availability of *global constraints*: high-level constraints such as AllDifferent (Régin 1994) or Regular (Pesant 2004) that capture structured combinatorial patterns and come with dedicated, efficient propagation algorithms (see (Beldiceanu, Carlsson, et al. 2007) for an extensive catalogue). We refer the reader to (Michel et al. 2021) for a comprehensive introduction to CP and CP solvers.

CP is particularly well-suited to scheduling, where the goal is to assign integer-valued start times, durations, and resource consumptions to a set of tasks subject to temporal and capacity restrictions (Baptiste et al. 2012; Laborie, Rogerie, Shaw, et al. 2018). The classical *cumulative* constraint (Aggoun and Beldiceanu 1993), for instance, takes a set of tasks each described by integer variables (s_i, d_i, c_i) for start time, duration, and resource consumption, and enforces that at every time point the total consumption of overlapping tasks does not exceed a fixed capacity. Similarly, the *disjunctive* constraint enforces that no two tasks overlap in time. These constraints, together with efficient filtering algorithms, form the backbone of CP-based scheduling solvers (Baptiste et al. 2012).

The modeling flexibility of CP for scheduling has been further enhanced with the introduction of *conditional time-interval variables* (Laborie and Rogerie 2008), which represent the execution of tasks that may or may not take place. Such tasks are called optional tasks. These are convenient for modeling problems with alternative resources (Cappart and Schaus 2017; Kinable et al. 2014; Kizilay et al. 2018; Kumar et al. 2018). Building on top of conditional time-intervals, the same authors introduced an algebraic model for cumulative resources called cumulative functions (Laborie, Rogerie, Shaw, et al. 2009). It provides an efficient and convenient way to model producer-consumer problems with optional tasks (Cappart, Thomas, et al. 2018; Gedik et al. 2018; Liu et al. 2018;

*Corresponding Author.

Authors' Contact Information: Pierre Schaus, ORCID: 0000-0002-3153-8941, pierre.schaus@uclouvain.be, UCLouvain, ICTEAM, Louvain-la-Neuve, Belgium; Charles Thomas, ORCID: 0000-0002-7360-5372, cftmthomas@gmail.com, UCLouvain, ICTEAM, Louvain-la-Neuve, Belgium; Roger Kameugne, ORCID: 0000-0003-1809-9822, rkameugne@gmail.com, UCLouvain, ICTEAM and University of Maroua, Louvain-la-Neuve / Maroua, Belgium / Cameroon.

Thomas and Schaus 2024). While commercial solvers such as CPOptimizer (Laborie, Rogerie, Shaw, et al. 2018) and OptalCP (Vilím and Pons 2023) natively support conditional time-intervals and cumulative functions, both are closed-source engines whose concrete propagation algorithms and data structures are proprietary. In the open-source domain, Gecode (Schulte et al. 2006) offers the generalized cumulative constraint (cumulatives) of Beldiceanu and Carlsson (2002). However, that constraint operates on flat arrays of integer variables rather than cumulative function expressions, encodes optionality indirectly through dummy resources, and relies on an event-sweep algorithm that exhibits several pruning limitations in practice. Conversely, profile-based timetabling algorithms (Gay et al. 2015a; Gingras and C. Quimper 2016) were developed strictly for the classical cumulative constraint with mandatory tasks and positive consumptions.

In this paper, we bridge this gap: (1) we provide an automated translation that flattens bound restrictions on cumulative functions into a generalized cumulative constraint; (2) we generalize profile-based timetabling to simultaneously handle conditional time-intervals, variable and negative consumptions (producers and consumers), and variable durations, achieving strictly stronger pruning than the sweep algorithm of Beldiceanu and Carlsson (2002); and (3) we contribute the first fully documented, open-source reference implementation of cumulative functions and generalized timetabling within the MaxiCP (Schaus et al. 2026) solver. The experimental results demonstrate that our algorithm achieves competitive filtering and scales effectively to large-scale scheduling problems.

The remainder of this article is organized as follows. Section 2 introduces the modeling background on conditional time-interval variables, cumulative functions, and the GeneralizedCumulative constraint. Section 3 presents our timetabling filtering algorithm, including timeline initialization, the filtering rules, and its theoretical complexity. Section 4 reviews related work and contrasts our approach with existing cumulative algorithms. Section 5 evaluates the performance of the algorithm on multiple benchmark scheduling problems across different solvers. Finally, Section 6 concludes and discusses avenues for future work.

2 Modeling with Cumulative Functions

This section introduces the modeling primitives used throughout the paper. We first define conditional time-interval variables and the elementary cumulative functions that describe a task’s contribution to a resource profile. Next, we show how to combine these functions to model complex resource profiles and flatten them into a set of cumulative tasks. Finally, we formally define the GeneralizedCumulative constraint and explain how restrictions on cumulative functions are enforced through it.

2.1 Conditional Time-Intervals

Following the original terminology introduced by Laborie and Rogerie (2008), a conditional time-interval variable is also called an *interval* for short.

DEFINITION 1 (CONDITIONAL TIME-INTERVAL VARIABLE). A conditional time-interval variable x , introduced by Laborie and Rogerie (2008), is a variable whose domain is a subset of $\{\perp\} \cup \{[s, e) \mid s \leq e, s, e \in \mathbb{Z}\}$. The value $x = \perp$ denotes that the interval is absent; the value $x = [s, e)$ denotes that it executes from s to e . Its domain is represented by a tuple $\langle s, d, e, p \rangle$ where:

- $p \in \{\text{true}, \text{false}\}$ is the execution status (false corresponds to $\perp$);
- $s \in [\underline{s}, \bar{s}]$ and $e \in [\underline{e}, \bar{e}]$ are the (non-negative) start and end time variables;
- $d \in [\underline{d}, \bar{d}]$ is the (non-negative) duration variable.

Bound-consistent filtering on $s + d = e$ is maintained atomically: if propagation reduces any of the ranges $[\underline{s}, \bar{s}]$, $[\underline{d}, \bar{d}]$, $[\underline{e}, \bar{e}]$ to empty (i.e. $\underline{s} > \bar{s}$, $\underline{d} > \bar{d}$, or $\underline{e} > \bar{e}$), the interval is forced absent ($p \leftarrow \text{false}$) if it is optional, or a contradiction is raised if $p = \text{true}$. When $p = \text{false}$, the interval is absent and the attributes s, d, e are irrelevant:

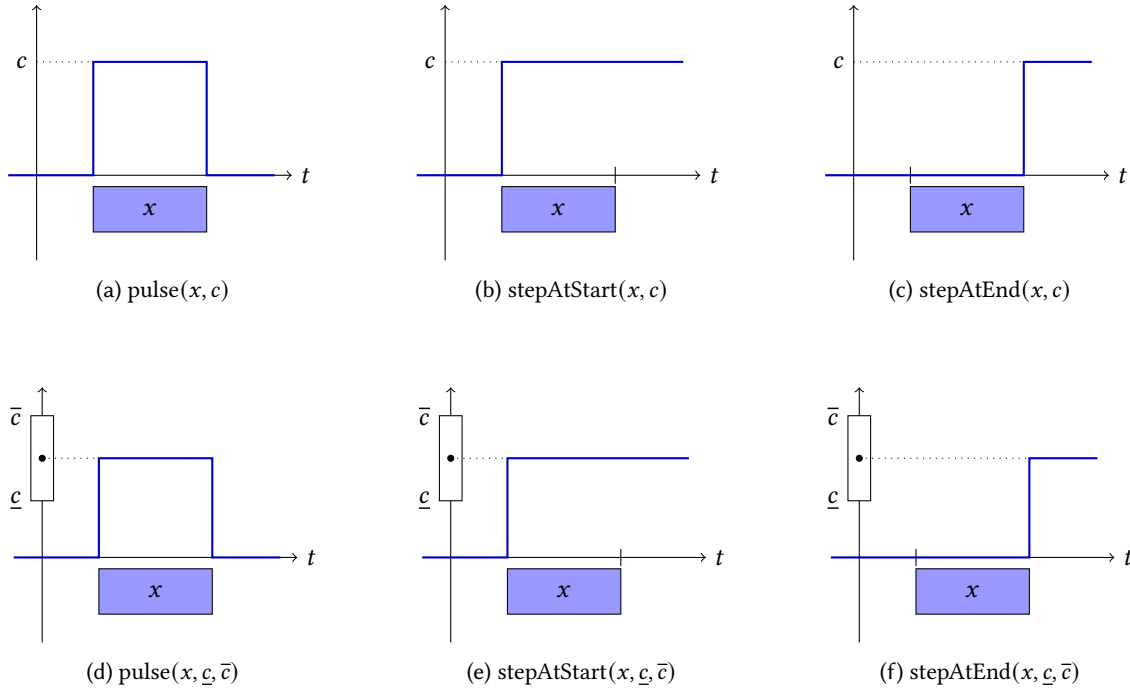


Fig. 1. Elementary cumulative function expressions with fixed consumption c (top row) and variable consumption range $[\underline{c}, \bar{c}]$ (bottom row) over interval x .

the interval contributes nothing to any constraint. We denote by I the set of all such intervals in the problem. For interval $i \in I$, its attributes are written s_i, d_i, e_i, p_i with bounds $\underline{s}_i, \bar{s}_i, \underline{d}_i, \bar{d}_i, \underline{e}_i, \bar{e}_i$.

2.2 Cumulative Functions

For modeling cumulative problems, Laborie, Rogerie, Shaw, et al. (2009) introduced the idea of cumulative function expressions. They allow the expression of step-wise integer functions of conditional time-interval variables with consumption on a resource. The contribution of a single time-interval variable $x \in I$ with a non-negative consumption $c \in \mathbb{Z}^+$ is represented by an elementary cumulative function. The elementary cumulative functions are *pulse*, *stepAtStart*, and *stepAtEnd* (Laborie, Rogerie, Shaw, et al. 2009) as illustrated in Figure 1.

DEFINITION 2 (ELEMENTARY CUMULATIVE FUNCTIONS). The following elementary cumulative functions (Laborie, Rogerie, Shaw, et al. 2009) express the contribution of a conditional time-interval variable x (with execution status p_x ,

start s_x , end e_x) and a fixed consumption $c \in \mathbb{Z}^+$ to a step-wise resource profile:

$$\begin{aligned} \text{pulse}(x, c)(t) &= \begin{cases} c & \text{if } p_x = \text{true and } s_x \leq t < e_x \\ 0 & \text{otherwise} \end{cases} \\ \text{stepAtStart}(x, c)(t) &= \begin{cases} c & \text{if } p_x = \text{true and } t \geq s_x \\ 0 & \text{otherwise} \end{cases} \\ \text{stepAtEnd}(x, c)(t) &= \begin{cases} c & \text{if } p_x = \text{true and } t \geq e_x \\ 0 & \text{otherwise} \end{cases} \end{aligned}$$

All three functions are step-wise integer functions that do not vary within an interval. When the consumption ranges over $[\underline{c}, \bar{c}]$ with $\underline{c} \leq \bar{c}$ and $\underline{c}, \bar{c} \in \mathbb{Z}^+$, the functions are defined analogously with a consumption variable $c \in [\underline{c}, \bar{c}]$. Figure 1 illustrates both variants (fixed consumption in the top row, consumption range in the bottom row) for each of the three functions. A deterministic step function for a fixed time point $t_0 \in \mathbb{Z}$ and $c \in \mathbb{Z}$ is also supported:

$$\text{step}(t_0, c)(t) = \begin{cases} c & \text{if } t \geq t_0 \\ 0 & \text{otherwise.} \end{cases}$$

Cumulative functions (elementary or not) can be composed using two binary operators. Given two cumulative functions f and g , the *plus* and *minus* operators are defined point-wise as:

$$\text{plus}(f, g)(t) = f(t) + g(t), \quad \text{minus}(f, g)(t) = f(t) - g(t).$$

A composite cumulative function can therefore be viewed as an Abstract Syntax Tree (AST) with elementary functions (*pulse*, *stepAtStart*, *stepAtEnd*, *step*) at the leaf nodes and *plus/minus* nodes as internal operators. Flattening a composite cumulative function produces a multiset of cumulative tasks (x', c') where $c' \in \mathbb{Z}$ is an integer consumption and x' is either the original interval (for *pulse*) or a fresh interval suitably constrained to the original interval (for *stepAtStart* and *stepAtEnd*).

DEFINITION 3 (CUMULATIVE TASK). A cumulative task is a pair (x, c) where $x \in I$ is a conditional time-interval variable and $c \in \mathbb{Z}$ is its consumption on the resource. We denote by T the set of cumulative tasks (or tasks for short) in the problem.

EXAMPLE 1. Let A , B , and C be three mandatory (with execution status true) conditional time-interval variables with fixed start times and durations (depicted as double-headed arrows in Figure 2, top right). They contribute to a cumulative function $f = \text{stepAtStart}(A, 2) - (\text{pulse}(B, 1) + \text{stepAtEnd}(C, 1))$. The resulting AST is shown on the left of Figure 2. On the bottom right, the colored border rectangles are the cumulative tasks $\{(A', 2), (B, -1), (C', -1)\}$ obtained by flattening f (see Algorithm 1); and the dotted step function is the resulting cumulative function f .

A cumulative function can be flattened using Algorithm 1 to extract a set of cumulative tasks, each represented as a pair (x, c) of an interval x and a consumption c (positive or negative). This algorithm traverses the AST and collects the intervals at the leaf nodes, reversing the consumption signs for the intervals flattened on the right branch of a minus node. Note that whenever encountering a *stepAtStart* (x, c) leaf node, the cumulative task collected is defined on a fresh interval x' related to the original one with: $x'.s = x.s$ (same start as x), $x'.p = x.p$ (same execution status), and $x'.e = \text{Horizon}$ (ending at the time horizon) to reflect the definition of a step function. Similarly, in the case of *stepAtEnd* (x, c) , a new interval x' with $x'.s = x.e$ (same start as end of x) is collected.

EXAMPLE 2. Continuing example 1, $\text{flatten}(f)$ collects the cumulative tasks $\{(A', 2), (B, -1), (C', -1)\}$ illustrated on the right of Figure 2.

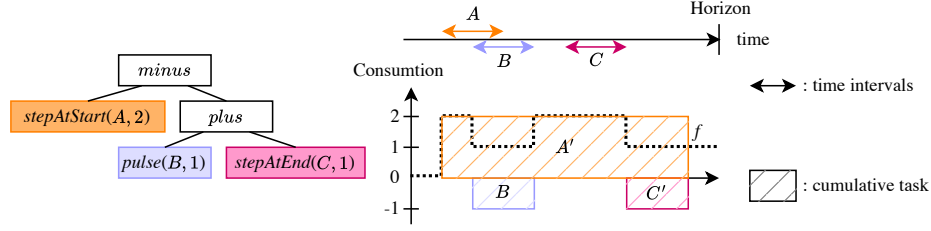


Fig. 2. Example 1: AST of f (left) and resource profile (right). Double-headed arrows (top) show the fixed time-intervals of A , B , C . Colored border rectangles represent the cumulative tasks $\{(A', 2), (B, -1), (C', -1)\}$ obtained by flattening f ; the dotted line is the resulting cumulative function f .

Algorithm 1: $\text{flatten}(n)$

Input: n : A node of the AST
Output: Set of cumulative tasks

```

1 if  $n = \text{pulse}(x, c)$  then return  $\{(x, c)\}$ 
2 else if  $n = \text{stepAtStart}(x, c)$  then
3    $x' \leftarrow \langle x.s, \text{Horizon} - x.s, \text{Horizon}, x.p \rangle$  // Fresh interval for stepAtStart
4   return  $\{(x', c)\}$ 
5 else if  $n = \text{stepAtEnd}(x, c)$  then
6    $x' \leftarrow \langle x.e, \text{Horizon} - x.e, \text{Horizon}, x.p \rangle$  // Fresh interval for stepAtEnd
7   return  $\{(x', c)\}$ 
8 else if  $n = \text{plus}(n_l, n_r)$  then return  $\text{flatten}(n_l) \cup \text{flatten}(n_r)$ 
9 else if  $n = \text{minus}(n_l, n_r)$  then return  $\text{flatten}(n_l) \cup \{(x, -c) \mid (x, c) \in \text{flatten}(n_r)\}$ 
10 else if  $n = \text{step}(t_0, c)$  then
11    $x' \leftarrow \langle t_0, \text{Horizon} - t_0, \text{Horizon}, \text{true} \rangle$  // Fresh interval for step
12   return  $\{(x', c)\}$ 
13 else error // Unknown pattern

```

2.3 Constraints on Cumulative Functions

The height of a cumulative function can be restricted with a lower and upper bound at every time overlapping at least one executing time-interval: $\underline{C} \leq f \leq \overline{C}$. This constraint is called *alwaysIn*($f, \underline{C}, \overline{C}$) in (Laborie, Rogerie, Shaw, et al. 2009). It can be enforced by posting a GeneralizedCumulative constraint requiring a resource consumption between $\underline{C}$ and $\overline{C}$ for the collection of cumulative tasks $T = \text{flatten}(f)$ (treated as a multiset preserving distinct task instances with identical parameters). The whole domain of a cumulative task $i \in T$ is encoded by a tuple of variables $\langle s_i, d_i, e_i, c_i, p_i \rangle$ where the integer variable c_i represents its consumption on the resource; c_i may be positive (producer), negative (consumer), or its domain $[\underline{c}_i, \overline{c}_i]$ may span both negative and positive values (i.e., $\underline{c}_i < 0 < \overline{c}_i$). The set of tasks T is partitioned into three subsets as given in Definition 4.

DEFINITION 4. The set $O = \{i \in T \mid p_i \in \{\text{true}, \text{false}\}\}$ denotes the set of optional tasks, those for which it is not decided yet if they will be executed or not on the resource. The set $R = \{i \in T \mid p_i = \text{true}\}$ denotes the set of required tasks, those which will surely be executed on the resource. The set $E = \{i \in T \mid p_i = \text{false}\}$ denotes the set of excluded tasks, those which will surely not be executed on the resource. The sets O , R and E are disjoint and form a partition of

the set T , i.e., $T = O \cup R \cup E$. A task i is fixed when its execution is required ($p_i = \text{true}$), its execution window is fixed ($\underline{s}_i = \bar{s}_i, \underline{e}_i = \bar{e}_i$), and its consumption is fixed ($\underline{c}_i = \bar{c}_i$); otherwise, it is non-fixed.

DEFINITION 5. The $\text{GeneralizedCumulative}(T, \underline{C}, \bar{C})$ constraint enforces the cumulated consumptions of the tasks of T to be within the fixed capacity range $[\underline{C}, \bar{C}]$ for every time at which at least one task of T is executed:

$$\sum_{i \in T | p_i = \text{true} \wedge s_i \leq \tau < e_i} c_i \in [\underline{C}, \bar{C}] \quad \forall \tau \in \bigcup_{i \in T | p_i = \text{true}} [s_i, e_i) \quad (1)$$

Restricted Time Interval Scope. To enforce the capacity range at every time point across the entire horizon rather than only when at least one task executes, one can add a dummy task that spans the entire horizon, with a mandatory execution status and a consumption of zero. More generally, the time scope can also be limited to a specific task, by linking the status variables of the tasks to the overlap of this task.

Consistency. Enforcing global bound-consistency for the cumulative constraint is NP-Hard (Garey and Johnson 1979). However, enforcing it on the decomposition into sum constraints at every point in the horizon, as in (1), is not NP-Hard. Unfortunately, this approach is inefficient for large horizons, as it requires a sum constraint to be enforced at every time point. Therefore, we aim to achieve the same filtering in polynomial time relative to the number of tasks, rather than the size of the horizon. This filtering is generally referred to as *timetabling* filtering. Intuitively, it works by aggregating all the fixed parts (when $\bar{s}_i < \underline{e}_i$) to create a pessimistic consumption profile. It then postpones the earliest execution of a task whenever it detects that starting it at this time would overload the resource capacity.

The next examples illustrate the modeling of two classical cumulative scheduling scenarios with cumulative functions and the $\text{GeneralizedCumulative}$ constraint.

EXAMPLE 3 (CLASSICAL CUMULATIVE CONSTRAINT). Consider n tasks $T = \{(x_i, c_i) \mid i = 1, \dots, n\}$, each with a fixed positive consumption $c_i > 0$, a mandatory execution ($p_i = \text{true}$), and a resource of capacity $C > 0$. The classical cumulative constraint requires that the total consumption of concurrently executing tasks never exceeds C . This is captured by the cumulative function

$$f = \sum_{i=1}^n \text{pulse}(x_i, c_i),$$

and posting $\text{GeneralizedCumulative}(\text{flatten}(f), -\infty, C)$ enforces that the total consumption stays under C at all times at which at least one task executes. Note that since f contains only pulse terms combined with $+$, the flatten function simply collects the pairs (x_i, c_i) without any sign reversal – it is essentially the identity in this case.

EXAMPLE 4 (PRODUCER/CONSUMER CONSTRAINT). Consider a reservoir (e.g., a tank or a battery) with level required to stay within $[L_{\min}, L_{\max}]$ at all times. A set of tasks $T = \{(x_i, c_i) \mid i = 1, \dots, n\}$ is defined, each with a fixed consumption c_i (positive or negative). Tasks that produce resources add to the level (positive consumption), while tasks that consume resources subtract from it (negative consumption). Let $P \subseteq T$ be the set of producer tasks, each with consumption $c_i > 0$, and $C \subseteq T$ be the set of consumer tasks, each with consumption $-c_j < 0$ ($c_j > 0$). Modeling the initial level ℓ_0 of the reservoir at time 0 can be done with a $\text{step}(0, \ell_0)$ function. The cumulative function

$$f = \text{step}(0, \ell_0) + \sum_{i \in P} \text{stepAtStart}(x_i, c_i) - \sum_{j \in C} \text{stepAtStart}(x_j, c_j)$$

represents the reservoir level over time. Posting $\text{GeneralizedCumulative}(\text{flatten}(f), L_{\min}, L_{\max})$ enforces that the level stays within $[L_{\min}, L_{\max}]$ at all times at which at least one task executes.

3 Timetabling Algorithm

This section first introduces the *Profile* data structure then explains the timetabling filtering algorithm for filtering the attributes of cumulative tasks.

3.1 The Profile Data Structure

The *Profile* data structure is an aggregation of adjoined rectangles of different durations and heights introduced by [Gingras and C. Quimper \(2016\)](#) and [Gay et al. \(2015a\)](#) to record the resource utilization of tasks. The end of a rectangle is the beginning of the next one. The starting (resp. ending) of a rectangle is represented by a tuple called *time point* with a component representing the beginning (resp. ending) time of the rectangle and other components that contain information relative to the resource consumption for the duration of the rectangle. These time points are sorted in increasing order of time and are kept in a doubly-linked list structure called the *Profile*, bounded by sentinel head and tail nodes with profile values of zero. An original idea of [Gingras and C. Quimper \(2016\)](#) is to have pointers $tpAt(\underline{s}_i)$ and $tpAt(\bar{e}_i)$ linking to the time point associated with $\underline{s}_i$ and $\bar{e}_i$ respectively.

3.2 Minimum and Maximum Profiles

To check and adjust the attributes of tasks, the *Profile* data structure from [Gingras and C. Quimper \(2016\)](#) is extended to compute both a minimum and maximum profile range at each time point. In our adaptation of the *Profile* data structure (hereafter referred to as *timeline* and noted $\mathcal{L}$), a time point $tp \in \mathcal{L}$ consists of a tuple $\langle time, \underline{P}, \bar{P}, \#fp \rangle$, where *time* corresponds to the start time of the time point, $\underline{P}$ (resp. $\bar{P}$) to the minimum (resp. maximum) resource profile of tasks over time and $\#fp$ to the number of fixed parts of tasks that overlap over *time*.

Intuitively, the minimum profile is obtained by considering the upper bound of the negative contribution and the lower bound of the positive contribution of each task. In contrast, the maximum profile considers the upper bound of the positive contribution and the lower bound of the negative contribution of each task. The upper bound of the negative (resp. positive) contribution of a task corresponds to its largest negative consumption in absolute terms, i.e., minimal algebraic value $\min(\underline{c}_i, 0)$ (resp. upper bound $\max(\bar{c}_i, 0)$), during the whole possible time window of the task, while the lower bound of its negative (resp. positive) contribution corresponds to its smallest negative consumption in absolute terms, i.e., $\min(\bar{c}_i, 0)$ (resp. lower bound $\max(\underline{c}_i, 0)$), during the fixed part of the task. More formally, the minimum (resp. maximum) resource profile at time point tp denoted $tp.\underline{P}$ (resp. $tp.\bar{P}$) is defined as:

$$tp.\underline{P} = \sum_{i \in O \cup R | \underline{s}_i \leq tp.time < \bar{e}_i} \min(\underline{c}_i, 0) + \sum_{i \in R | \bar{s}_i \leq tp.time < \underline{e}_i} \max(\bar{c}_i, 0), \quad (2)$$

$$tp.\bar{P} = \sum_{i \in O \cup R | \underline{s}_i \leq tp.time < \bar{e}_i} \max(\bar{c}_i, 0) + \sum_{i \in R | \bar{s}_i \leq tp.time < \underline{e}_i} \min(\bar{c}_i, 0), \quad (3)$$

where $\{i \in O \cup R | \underline{s}_i \leq tp.time < \bar{e}_i\}$ corresponds to the set of all optional and required tasks whose time window overlaps the time $tp.time$ and $\{i \in R | \bar{s}_i \leq tp.time < \underline{e}_i\}$ corresponds to the set of all required tasks that have a fixed part that overlaps the time $tp.time$.

The proposed timetabling filtering algorithm is split into two steps: First, the timeline $\mathcal{L}$ is initialized by creating and ordering the time points, linking them with the tasks, computing their minimum and maximum profile ranges, and checking the feasibility consistency by comparing the profile ranges with the maximum and minimum capacity. Second, each task is checked during its entire possible span in terms of profile and capacity to adjust its domain.

3.3 Timeline Initialization and Feasibility Consistency Check

The function *initializeTimeline* (Algorithm 2) initializes the timeline, links the corresponding time points to tasks, and computes the minimum and maximum profile of the resource utilization. It receives as input the set of tasks T and the bounds of the resource capacity $[\underline{C}, \overline{C}]$. First, events are generated by iterating over the tasks that are not absent (Line 2). An event associated with a task i is a triple $e = \langle \text{time}, \text{type}, \text{task} \rangle$ where $e.\text{time} \in \{\underline{s}_i, \overline{s}_i, \underline{e}_i, \overline{e}_i \mid i \in T\}$ is the time of the event, $e.\text{type} \in \{\underline{s}, \overline{s}, \underline{e}, \overline{e}\}$ is the type of the event (start min, start max, end min, or end max), and $e.\text{task} = i$ is the index of the task. Each non-absent task generates a maximum of four events, two for the start and end of its possible time window and two for the start and end of its fixed part if it has one. For each non-absent task i , two events are generated which correspond to the minimum start time $(\underline{s}_i, \underline{s}, i)$, and the maximum end time $(\overline{e}_i, \overline{e}, i)$ of the task. For tasks that are present and with a fixed part, two additional events are generated for the maximum start time $(\overline{s}_i, \overline{s}, i)$ and the minimum end time $(\underline{e}_i, \underline{e}, i)$ of the task.

A current time point tp is maintained and corresponds to the last time point of the timeline being initialized. Then, these events are processed by order of increasing time to initialize the timeline $\mathcal{L}$ (Line 10). New time points are created only when the time of the event differs from the current time point tp as checked at Line 13. This means that if several events share the same time, they will all contribute towards the same time point; because accumulator additions and subtractions are commutative, any tie-breaking order among events that share the same time yields the identical time point profile. When a new time point is created, it is appended at the end of the timeline which links it to the previous last time point. When processing a start min ($\underline{s}$) or end max ($\overline{e}$) event, the current time point is linked to the task event at Lines 21 and 25 to retrieve the time points corresponding to the start and end of tasks in constant time later.

Three accumulators are used to keep track of the minimum (P) and maximum ($\overline{P}$) profile as well as the number of fixed parts of tasks overlapping the current time point ($\#fp$). Depending on the nature of the event, the positive or negative contribution of its associated task is either added to (in case of $\underline{s}$ or $\overline{s}$) or subtracted (in case of $\overline{e}$ or $\underline{e}$) from the accumulators and the current time point tp is updated.

As the timeline is initialized, the feasibility consistency of the minimum and maximum profiles is checked for each time point at Line 14. Once a time point is completed (when creating the next time point), we ensure that if at least one task is guaranteed to execute during the time point ($\#fp > 0$), the minimum profile $tp.P$ is not above the maximum capacity of the resource $\overline{C}$ and the maximum profile $tp.\overline{P}$ is not below the minimum capacity of the resource $\underline{C}$. At the last time point of the timeline, no task is guaranteed to execute, so the consistency check is not performed. The correctness of the profile construction and the soundness of the infeasibility check are established in the following lemma and theorem (proofs are deferred to Appendix A).

LEMMA 1 (CORRECTNESS OF THE PROFILE BOUNDS). *For every time point $tp \in \mathcal{L}$, immediately after processing all events whose time is at most $tp.\text{time}$, the accumulator values $tp.P$ and $tp.\overline{P}$ satisfy (2) and (3), respectively, and the number of overlapping fixed parts is exactly*

$$tp.\#fp = |\{i \in R \mid \overline{s}_i \leq tp.\text{time} < \underline{e}_i\}|.$$

THEOREM 2 (SOUNDNESS OF THE INFEASIBILITY CHECK). *If Algorithm 2 returns Fail at Line 15, then the current domains of the tasks are inconsistent with the GeneralizedCumulative constraint.*

The complexity of the *initializeTimeline* function is given in Theorem 3.

THEOREM 3 (COMPLEXITY OF initializeTimeline). *Algorithm 2 has a worst-case time complexity of $O(n \log n)$, where $n = |T|$.*

Example 5 illustrates the profile initialization for three tasks.

Algorithm 2: initializeTimeline($T, \underline{C}, \overline{C}$)

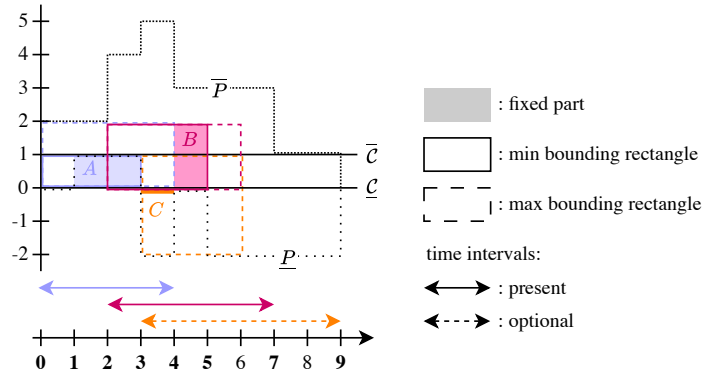
```

1  events  $\leftarrow \{\}$  // Initialize the set of events
2  for  $i \in \{j \in T \mid p_j \neq \text{false}\}$  do
3      events  $\leftarrow \text{events} \cup \{(\underline{s}_i, \underline{s}, i), (\overline{e}_i, \overline{e}, i)\}$  // Each event is a (time, type, task) triple
4      if  $p_i = \text{true} \wedge \overline{s}_i < \underline{e}_i$  then
5          events  $\leftarrow \text{events} \cup \{(\overline{s}_i, \overline{s}, i), (\underline{e}_i, \underline{e}, i)\}$  // Add fixed-part events for required tasks
6   $\underline{P} \leftarrow 0; \overline{P} \leftarrow 0$  // Lower/Upper bound of the profile at current time point
7   $\#fp \leftarrow 0$  // Number of overlapping fixed parts of current time point
8  time  $\leftarrow \min_{e \in \text{events}} e.\text{time}$  // Initialize current time to the earliest event time
9   $tp \leftarrow \langle \text{time}, \underline{P}, \overline{P}, \#fp \rangle$  // Current time point; a timepoint record stores  $\{\text{time}, \underline{P}, \overline{P}, \#fp\}$ 
10  $\mathcal{L} \leftarrow \{tp\}$  // All time points created so-far
11 for  $e \in \text{events}$  sorted by time do
12      $i \leftarrow e.\text{task}$ 
13     if  $tp.\text{time} < e.\text{time}$  then
14         if  $\#fp > 0 \wedge (\underline{P} > \overline{C} \vee \overline{P} < \underline{C})$  then
15             return Fail // Capacity violation detected
16              $tp \leftarrow \langle e.\text{time}, \underline{P}, \overline{P}, \#fp \rangle$  // New time point
17              $\mathcal{L}.\text{append}(tp)$ 
18     if  $e.\text{type} = \underline{s}$  then // Process earliest start event
19          $\underline{P} \leftarrow \underline{P} + \min(\underline{c}_i, 0)$  // Add pessimistic negative contribution
20          $\overline{P} \leftarrow \overline{P} + \max(\overline{c}_i, 0)$  // Add optimistic positive contribution
21          $tpAt(\underline{s}_i) \leftarrow tp$  // Store time-point pointer for  $\underline{s}_i$ 
22     if  $e.\text{type} = \overline{e}$  then // Process latest end event
23          $\underline{P} \leftarrow \underline{P} - \min(\underline{c}_i, 0)$  // Remove pessimistic negative contribution
24          $\overline{P} \leftarrow \overline{P} - \max(\overline{c}_i, 0)$  // Remove optimistic positive contribution
25          $tpAt(\overline{e}_i) \leftarrow tp$  // Store time-point pointer for  $\overline{e}_i$ 
26     if  $p_i = \text{true} \wedge \overline{s}_i < \underline{e}_i$  then
27         if  $e.\text{type} = \overline{s}$  then // Process start of fixed parts event
28              $\underline{P} \leftarrow \underline{P} + \max(\underline{c}_i, 0)$  // Add pessimistic positive contribution
29              $\overline{P} \leftarrow \overline{P} + \min(\overline{c}_i, 0)$  // Add optimistic negative contribution
30              $\#fp \leftarrow \#fp + 1$  // Increment overlapping fixed part counter
31         if  $e.\text{type} = \underline{e}$  then // Process end of fixed parts event
32              $\underline{P} \leftarrow \underline{P} - \max(\underline{c}_i, 0)$  // Remove pessimistic positive contribution
33              $\overline{P} \leftarrow \overline{P} - \min(\overline{c}_i, 0)$  // Remove optimistic negative contribution
34              $\#fp \leftarrow \#fp - 1$  // Decrement overlapping fixed part counter
35      $tp.\underline{P} \leftarrow \underline{P}; tp.\overline{P} \leftarrow \overline{P}; tp.\#fp \leftarrow \#fp$  // Update current time point attributes
36 return  $\mathcal{L}$  // Return the complete timeline

```

<i>time</i>	<i>event(s)</i>	$\underline{P}$	$\overline{P}$	$\#fp$
0	$\underline{s}_A$	0	2	0
1	$\overline{s}_A$	1	2	1
2	$\underline{s}_B$	1	4	1
3	$\underline{e}_A, \underline{s}_C$	-2	5	0
4	$\overline{e}_A, \overline{s}_B$	0	3	1
5	$\underline{e}_B$	-2	3	0
7	$\overline{e}_B$	-2	1	0
9	$\overline{e}_C$	0	0	0

Table 1. Time points computed from the tasks of Example 5.

Fig. 3. Representation of the tasks of Example 5 with the resulting minimum ($\underline{P}$) and maximum ($\overline{P}$) profile. The minimum and maximum bounding rectangles of each task are also represented.

EXAMPLE 5. Let us consider three tasks :

$A : \langle s = [0, 1], d = [3, 4], e = [3, 4], c = [1, 2], p = \text{true} \rangle;$

$B : \langle s = [2, 4], d = [3, 4], e = [5, 7], c = 2, p = \text{true} \rangle;$

$C : \langle s = [3, 8], d = [1, 3], e = [4, 9], c = [-2, 1], p = \{\text{true}, \text{false}\} \rangle;$

and a resource of capacity bounds $[0, 1]$. The time points and their attributes are shown in Table 1.

Note that the events $\overline{s}_C$ and $\underline{e}_C$ are not considered because task C is optional and has no fixed part ($\overline{s}_C \geq \underline{e}_C$). Figure 3 shows a representation of the tasks and the resulting profile ranges.

3.4 Filtering

We first introduce a formal notion of timetabling filtering consistency for the GeneralizedCumulative constraint, and then present the four filtering rules whose purpose is to achieve it. Recall that a task $i \in T$ is fixed when $p_i = \text{true}$, $\underline{s}_i = \overline{s}_i$, $\underline{e}_i = \overline{e}_i$, and $\underline{c}_i = \overline{c}_i$ (Definition 4); otherwise, it is non-fixed. Timetabling filtering operates with respect to the timeline profile $\mathcal{L}$ constructed once per propagation cycle by Algorithm 2. When $\mathcal{L}$ is computed from the initial task domains, tasks are partitioned according to their presence status at that moment into initially required tasks $R_{\text{init}} = \{i \in T \mid p_i = \text{true}\}$ and initially optional tasks $O_{\text{init}} = \{i \in T \mid p_i \in \{\text{true}, \text{false}\}\}$. Crucially,

in Algorithm 2 (Line 27), only tasks that are already required at initialization ($i \in R_{\text{init}}$) have their fixed part compiled into the profile $\mathcal{L}$; optional tasks do not. Because the timeline $\mathcal{L}$ remains static throughout the filtering pass, whether a task's fixed part is embedded in $\mathcal{L}$ is determined once and for all by its membership in R_{init} or O_{init} , even if an optional task becomes mandatory during propagation. This distinction is necessary to evaluate the residual capacity available to a task without double counting.

DEFINITION 6 (CONFLICTING TIME POINTS). *Let us assume that the current task domains are feasibly consistent with the capacity constraints, let $\mathcal{L}$ be a timeline computed from the current task domains, and let $i \in R_{\text{init}} \cup O_{\text{init}}$ be a non-fixed task. We denote by fp_i the fixed part of task i :*

$$fp_i = \begin{cases} [\bar{s}_i, \underline{e}_i) & \text{if } i \in R_{\text{init}} \text{ and } \bar{s}_i < \underline{e}_i, \\ \emptyset & \text{otherwise.} \end{cases}$$

A time point $tp \in \mathcal{L}$ is said to be conflicting with i if it satisfies at least one of the following conditions:

(C1) Forward-conflicting: $tp.time \in [\underline{s}_i, \min(\bar{s}_i, \underline{e}_i))$ and

$$tp.\underline{P} + \max(\underline{c}_i, 0) > \bar{C} \quad \vee \quad tp.\bar{P} + \min(\bar{c}_i, 0) < \underline{C}.$$

(C2) Backward-conflicting: $tp.time \in [\max(\bar{s}_i, \underline{e}_i), \bar{e}_i)$ and

$$tp.\underline{P} + \max(\underline{c}_i, 0) > \bar{C} \quad \vee \quad tp.\bar{P} + \min(\bar{c}_i, 0) < \underline{C}.$$

(C3) Mandatory-conflicting: $tp.time \in [\underline{s}_i, \bar{e}_i) \setminus fp_i$ with $tp.\#fp > 0$, and

$$tp.\underline{P} - \min(\underline{c}_i, 0) > \bar{C} \quad \vee \quad tp.\bar{P} - \max(\bar{c}_i, 0) < \underline{C}.$$

(C4) Overlap consumption-conflicting: $\bar{s}_i < \underline{e}_i$, $tp.time \in [\bar{s}_i, \underline{e}_i)$, and

$$\begin{cases} \bar{c}_i > \bar{C} - (tp.\underline{P} - \underline{c}_i) \vee \underline{c}_i < \underline{C} - (tp.\bar{P} - \bar{c}_i), & \text{if } i \in R_{\text{init}} (tp.time \in fp_i), \\ \bar{c}_i > \bar{C} - (tp.\underline{P} - \min(\underline{c}_i, 0)) \vee \underline{c}_i < \underline{C} - (tp.\bar{P} - \max(\bar{c}_i, 0)), & \text{if } i \in O_{\text{init}}. \end{cases}$$

Intuitively, conditions (C1) and (C2) identify time points where scheduling task i at its earliest start or latest end would violate capacity; (C3) identifies time points outside the fixed part where omitting task i causes an unavoidable capacity violation, requiring i to be present; and (C4) restricts the consumption of task i over the interval $[\bar{s}_i, \underline{e}_i)$ during which it necessarily executes if present. When $i \in R_{\text{init}}$, this interval is the fixed part fp_i of task i and was compiled into the profile $\mathcal{L}$, so its full contribution is removed; when $i \in O_{\text{init}}$, task i has no fixed part in $\mathcal{L}$ ($fp_i = \emptyset$) and did not contribute to $\mathcal{L}$ with a fixed part, so only its non-fixed contribution is removed to avoid double counting.

DEFINITION 7 (TIMETABLING FILTERING CONSISTENCY). *The constraint GeneralizedCumulative($T, \underline{C}, \bar{C}$) is timetabling filtering consistent with respect to $\mathcal{L}$ if every non-fixed task $i \in R_{\text{init}} \cup O_{\text{init}}$ satisfies the following conditions:*

(TC1) Pointwise consistency: No time point $tp \in \mathcal{L}$ is conflicting with i (of type (C1) through (C4) in Definition 6).

(TC2) Free-part consumption consistency: If $\bar{s}_i \geq \underline{e}_i$ and $\underline{d}_i \geq 1$, its consumption bounds satisfy:

$$\begin{aligned} \bar{c}_i &\leq \max_{tp \in \mathcal{L} | \underline{e}_i - 1 < tp.next.time \wedge tp.time \leq \bar{s}_i} (\bar{C} - (tp.\underline{P} - \min(\underline{c}_i, 0))), \\ \underline{c}_i &\geq \min_{tp \in \mathcal{L} | \underline{e}_i - 1 < tp.next.time \wedge tp.time \leq \bar{s}_i} (\underline{C} - (tp.\bar{P} - \max(\bar{c}_i, 0))). \end{aligned}$$

(TC3) Duration consistency: If $\bar{s}_i \geq \underline{e}_i$, its maximum duration satisfies $\bar{d}_i \leq \max_{[a,b) \in A} (b - a)$, where

$$A = \{[a, b) \subseteq [\underline{s}_i, \bar{e}_i) \mid \forall tp \in \mathcal{L} \text{ with } tp.time \in [a, b), tp \text{ is not (C1)- or (C2)-conflicting with } i\}.$$

Intuitively, condition **(TC1)** requires all local time-window, presence, and fixed-part consumption conflicts (Definition 6) to be eliminated. Condition **(TC2)** applies to tasks without a fixed part ($\bar{s}_i \geq \underline{e}_i$): because any execution of duration $\underline{d}_i \geq 1$ necessarily intersects the core interval $[\underline{e}_i - 1, \bar{s}_i]$, its consumption must be compatible with at least one point in this interval, bounding $[\underline{c}_i, \bar{c}_i]$ by the available residual capacity. Finally, condition **(TC3)** reflects that task i cannot overlap any (C1)- or (C2)-conflicting time point; thus, every feasible execution must lie entirely within a conflict-free window $[a, b) \in A$, bounding the maximum duration $\bar{d}_i$ by the longest such window.

The four filtering rules below show how to enforce the conditions of Definitions 6 and 7: the *Forbid* rule (enforcing **(C1)** and **(C2)**), the *Mandatory* rule (enforcing **(C3)**), and the overlap case of the *Consumption* rule (enforcing **(C4)**) together achieve pointwise consistency **(TC1)**; the free-part case of the *Consumption* rule achieves free-part consumption consistency **(TC2)**; and the *Duration* rule achieves duration consistency **(TC3)**.

Forbid. The first rule is used to reduce the time windows of tasks by checking for each task $i \in T$ if starting (resp. ending) the task at its minimum start time (resp. maximum end time) is possible in regards to the profile range and the capacity range. If this is not the case, the task is pushed to the earliest (resp. latest) time at which it can be placed without violating the capacity range. Formally, the rule is defined as:

$$tp.\underline{P} + \max(\underline{c}_i, 0) > \bar{C} \vee tp.\bar{P} + \min(\bar{c}_i, 0) < \underline{C} \implies \underline{s}_i \geq tp.next.time, \\ \forall tp \in \mathcal{L} \text{ such that } \underline{s}_i \leq tp.time < \min(\bar{s}_i, \underline{e}_i) \quad (4)$$

$$tp.\underline{P} + \max(\underline{c}_i, 0) > \bar{C} \vee tp.\bar{P} + \min(\bar{c}_i, 0) < \underline{C} \implies \bar{e}_i \leq tp.time, \\ \forall tp \in \mathcal{L} \text{ such that } \max(\bar{s}_i, \underline{e}_i) \leq tp.time < \bar{e}_i \quad (5)$$

Note that as tasks are modeled with interval variables, the rule $e_i = s_i + d_i$ is enforced internally in the variable. Thus, when a task i has its minimum start time $\underline{s}_i$ (resp. maximum end time $\bar{e}_i$) adjusted, its minimum end time $\underline{e}_i$ (resp. maximum start time $\bar{s}_i$) is also updated by the relation $\underline{e}_i \geq \underline{s}_i + \underline{d}_i$ (resp. $\bar{s}_i \leq \bar{e}_i - \underline{d}_i$) embedded in the interval variable.

Mandatory. The second rule detects if the execution of a task $i \in T$ at a time point tp is necessary to avoid a violation of the resource capacity range at this time point. If this is the case, the task is adjusted to make sure it is present during the time point:

- (1) if the task is optional, it is set to present;
- (2) its time window is adjusted such that it executes over the whole time point;
- (3) its consumption is adjusted to avoid the profile range violation.

In formal terms, the rule is defined as:

$$tp.\underline{P} - \min(\underline{c}_i, 0) > \bar{C} \vee tp.\bar{P} - \max(\bar{c}_i, 0) < \underline{C} \\ \implies \begin{cases} p_i \leftarrow true, \\ \bar{s}_i \leq tp.time, \quad \underline{e}_i \geq tp.next.time, \\ \underline{c}_i \geq deficit \quad (\text{if } deficit > 0), \\ \bar{c}_i \leq overload \quad (\text{if } overload < 0) \end{cases} \quad (6) \\ \forall tp \in \mathcal{L} \text{ such that } tp.time \in [\underline{s}_i, \bar{e}_i) \setminus fp_i \wedge tp.\#fp > 0$$

where $deficit = \underline{C} - (tp.\bar{P} - \max(\bar{c}_i, 0))$ and $overload = \bar{C} - (tp.\underline{P} - \min(\underline{c}_i, 0))$ are the deficit and overload of consumption needed for the profile range to intersect the capacity range.

Consumption. This rule adjusts the consumption of each task $i \in T$ in regard to the resource capacity range. The maximum (resp. minimum) value of the consumption of a task is bounded by the difference between the maximum (resp. minimum) capacity of the resource and the minimum (resp. maximum) profile without the contribution of the task. Two cases are possible:

- (1) If the task has $\bar{s}_i < \underline{e}_i$, its consumption is checked and adjusted directly at each time point in $[\bar{s}_i, \underline{e}_i]$:

$$\begin{aligned} \bar{c}_i &\leq \bar{C} - (tp.\underline{P} - \underline{c}_i) \\ \underline{c}_i &\geq \underline{C} - (tp.\bar{P} - \bar{c}_i) \end{aligned} \quad \forall tp \in \mathcal{L} : tp.time \in fp_i \quad (i \in R_{init}) \quad (7)$$

$$\begin{aligned} \bar{c}_i &\leq \bar{C} - (tp.\underline{P} - \min(\underline{c}_i, 0)) \\ \underline{c}_i &\geq \underline{C} - (tp.\bar{P} - \max(\bar{c}_i, 0)) \end{aligned} \quad \forall tp \in \mathcal{L} : tp.time \in [\bar{s}_i, \underline{e}_i] \quad (i \in O_{init}) \quad (8)$$

If the task was initially required ($i \in R_{init}$), this interval is its fixed part fp_i and contributed to the profile, so its full contribution is removed before the residual capacity is checked; if it was initially optional ($i \in O_{init}$), it has no fixed part in $\mathcal{L}$ ($fp_i = \emptyset$) and did not contribute to $\mathcal{L}$ with a fixed part, so only its non-fixed contribution is removed to avoid double counting.

- (2) If the task has $\bar{s}_i \geq \underline{e}_i$ (free part) and a strictly positive minimal duration ($\underline{d}_i \geq 1$), the maximum available consumption over the minimum overlapping interval of the task is used to adjust its consumption:

$$\bar{c}_i \leq \max_{tp \in \{tp \in \mathcal{L} | \underline{e}_i - 1 < tp.next.time \wedge tp.time \leq \bar{s}_i\}} (\bar{C} - (tp.\underline{P} - \min(\underline{c}_i, 0))) \quad (9)$$

$$\underline{c}_i \geq \min_{tp \in \{tp \in \mathcal{L} | \underline{e}_i - 1 < tp.next.time \wedge tp.time \leq \bar{s}_i\}} (\underline{C} - (tp.\bar{P} - \max(\bar{c}_i, 0))) \quad (10)$$

The minimum overlapping interval was introduced by Gay et al. (2015b) and is defined for a task $i \in T$ with $\underline{d}_i \geq 1$ as the interval $[\underline{e}_i - 1, \bar{s}_i]$. Intuitively, it corresponds to the smallest interval such that, no matter its start time, duration or end time, the task executes during at least one time unit of this interval. (If $\underline{d}_i = 0$, the task may execute with duration zero without consuming any resource, so its consumption is not restricted by this rule.)

Duration. The last rule is inspired by the approach of Y. Ouellet and C.-G. Quimper (2019). It adjusts the maximum duration $\bar{d}$ of the tasks that have $\bar{s}_i \geq \underline{e}_i$ (for tasks with $\bar{s}_i < \underline{e}_i$, the adjustment would be redundant with the *Forbid* rule). The principle is to find the longest time span during which the task can be scheduled without a resource capacity violation. Again, we rely on the internal propagation rules of the interval variables to set the task as absent if the adjustment would empty its domain and to update the $\underline{s}_i$ and $\bar{e}_i$ attributes following the relations $\underline{s}_i \geq \underline{e}_i - \bar{d}_i$ and $\bar{e}_i \leq \bar{s}_i + \bar{d}_i$. Formally, the rule is written as:

$$\bar{d}_i \leq \max_{[a,b] \in A} (b - a), \quad (11)$$

where

$$A = \left\{ [a, b) \subseteq [\underline{s}_i, \bar{e}_i) \mid \begin{array}{l} \forall tp \in \mathcal{L} \text{ with } tp.time \in [a, b), \\ tp.\underline{P} + \max(\underline{c}_i, 0) \leq \bar{C} \wedge tp.\bar{P} + \min(\bar{c}_i, 0) \geq \underline{C} \end{array} \right\}.$$

Intuitively, A is the set of intervals where the task can be scheduled without underloading the minimum and overloading maximum capacity.

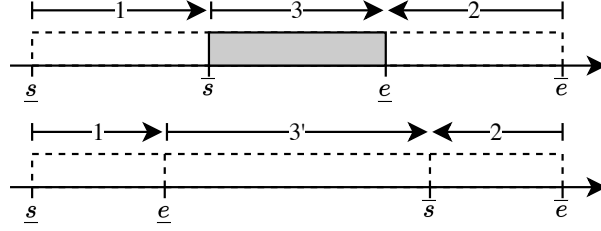


Fig. 4. Timetabling processing order for a task with a fixed part (top) and one without (bottom). Those steps (1, 2, 3, and 3') are also referenced in Algorithm 3 comments.

Algorithm. The function *timetabling* of Algorithm 3 enforces these rules for each task $i \in T$. This algorithm receives as input the set of tasks T and the bounds of the resource capacity $[\underline{C}, \bar{C}]$. The algorithm initializes the timeline with Algorithm 2 and iterates over all non-fixed tasks at Line 3. Each task is processed in three steps:

First (step 1), the *Forbid* rule is used to adjust the minimum start time of the task by iterating forward over the time points in the interval $[\underline{s}_i, \min(\bar{s}_i, \underline{e}_i))$ in the loop at Line 6. Second (step 2), the *Forbid* rule is used to adjust the maximum end time of the task by iterating backward over the interval $[\max(\bar{s}_i, \underline{e}_i), \bar{e}_i)$ in the loop at Line 11.

The last step iterates over the time points of the remaining part of the time window, depending on whether $\bar{s}_i < \underline{e}_i$. In the former case (step 3), the loop at Line 17 iterates over $[\bar{s}_i, \underline{e}_i)$: for tasks initially required ($i \in R_{\text{init}}$), this interval is its fixed part fp_i and its consumption is adjusted following the 1st case of the *Consumption* rule; for tasks initially optional ($i \in O_{\text{init}}$), it has no fixed part in $\mathcal{L}$, so *checkIfMandatory* is evaluated at each time point to test whether the task must become mandatory. In the latter case where $\bar{s}_i \geq \underline{e}_i$ (step 3'), the loop at Line 28 is executed. The consumption of the task is adjusted according to the 2nd case of the *Consumption* rule and its maximum duration is adjusted following the *Duration* rule. These three steps are illustrated in Figure 4.

In order to compute the maximum duration of the task, we maintain the variables $\bar{d}^*$ which contains the maximum duration found so far and s' which corresponds to the earliest time at which the task can start and span without capacity violation until the current time point. During the loop, $\bar{d}^*$ is updated if the difference between the current time point time and s' is greater than its previous value. If the task is detected as infeasible during a time point, s' is set to the end of this time point (Line 30). Note that in order to take into account the parts of the time window that are not considered in Loop 28, s' is initialized to $\underline{s}$ and the difference between $\bar{e}$ and s' is considered at the end of the loop (Line 39).

Similarly, when $\underline{d}_i \geq 1$, the minimum and maximum available consumptions are computed as $\underline{c}^*$ and $\bar{c}^*$ and used to update the consumption of the task if it has no fixed part (Line 38). Note that these variables are initialized based on the consumption available at the time point before the start of Loop 28 (tpf_{prev}) in order to consider the whole minimum overlapping interval of the task, and if a time point starts at $\bar{s}_i$, it is evaluated at the end of the scan to cover the upper boundary of the minimum overlapping interval; if $\underline{d}_i = 0$, the task may execute with duration zero, so consumption is not restricted.

The *Mandatory* rule is enforced over the time window outside the fixed part of each task. The function *checkIfMandatory* of Algorithm 4 is used to check this rule and apply its adjustments if needed. For a required task ($i \in R$), this check is not needed over its fixed part $[\bar{s}_i, \underline{e}_i)$ because the task is already established to execute across this interval, and its consumption is adjusted directly in Step 3. If the task is optional ($i \in O$), its execution over $[\bar{s}_i, \underline{e}_i)$ is not yet guaranteed, so *checkIfMandatory* is evaluated at each time point in Step 3 at Line 18. Across all branches, *checkIfMandatory* is called at Lines 8, 13, 18, and 31.

Note that some optimizations are possible depending on the set of tasks given as input. Indeed, the *Mandatory* propagation rule is only useful if the tasks have a mix of positive and negative consumptions or if there is a

Algorithm 3: timetabling($T, \underline{C}, \overline{C}$)

```

1  $\mathcal{L} \leftarrow \text{initializeTimeline}(T, \underline{C}, \overline{C})$  // Build timeline profile from initial domains
2  $R_{\text{init}} \leftarrow \{i \in T \mid p_i = \text{true}\}; O_{\text{init}} \leftarrow \{i \in T \mid p_i \in \{\text{true}, \text{false}\}\}$  // Initial Required and Optional tasks
3 for  $i \in R_{\text{init}} \cup O_{\text{init}}$  do
4   if  $i$  is fixed  $\vee \overline{d}_i = 0$  then continue // Skip fixed or 0-duration tasks
5    $tpf \leftarrow tpAt(\underline{s}_i)$  // First time point at or after  $\underline{s}_i$ 
6   while  $tpf.time < \min(\overline{s}_i, \underline{e}_i)$  do // Step 1: forward scan to adjust  $\underline{s}_i$ 
7     if  $tpf.\underline{P} + \max(\underline{c}_i, 0) > \overline{C} \vee tpf.\overline{P} + \min(\overline{c}_i, 0) < \underline{C}$  then  $\underline{s}_i \leftarrow \max(tpf.next.time, \underline{s}_i)$ 
8     else checkIfMandatory( $i, tpf, \underline{C}, \overline{C}$ ) // Check mandatory condition (C3)
9      $tpf \leftarrow tpf.next$ 
10   $tpb \leftarrow tpAt(\overline{e}_i).prev$  // Last time point before  $\overline{e}_i$ 
11  while  $tpb.time \geq \max(\overline{s}_i, \underline{e}_i)$  do // Step 2: backward scan to adjust  $\overline{e}_i$ 
12    if  $tpb.\underline{P} + \max(\underline{c}_i, 0) > \overline{C} \vee tpb.\overline{P} + \min(\overline{c}_i, 0) < \underline{C}$  then  $\overline{e}_i \leftarrow \min(tpb.time, \overline{e}_i)$ 
13    else checkIfMandatory( $i, tpb, \underline{C}, \overline{C}$ ) // Check mandatory condition (C3)
14     $tpb \leftarrow tpb.prev$ 
15  if  $\overline{s}_i < \underline{e}_i$  then // Step 3: task with fixed part ( $\overline{s}_i < \underline{e}_i$ )
16     $c_i^{\max} \leftarrow \overline{c}_i; c_i^{\min} \leftarrow \underline{c}_i$  // Save bounds for this scan
17    while  $tpf.time < \underline{e}_i$  do // Scan overlapping interval  $[\overline{s}_i, \underline{e}_i]$ 
18      if  $i \in O_{\text{init}}$  then checkIfMandatory( $i, tpf, \underline{C}, \overline{C}$ )
19      if  $i \in R_{\text{init}}$  then // Required: fixed part contributed to  $\mathcal{L}$ 
20         $\underline{c}_i \leftarrow \max(\underline{C} - (tpf.\overline{P} - c_i^{\max}), \underline{c}_i); \overline{c}_i \leftarrow \min(\overline{C} - (tpf.\underline{P} - c_i^{\min}), \overline{c}_i)$ 
21      else // Optional: no fixed part in  $\mathcal{L}$ 
22         $\underline{c}_i \leftarrow \max(\underline{C} - (tpf.\overline{P} - \max(c_i^{\max}, 0)), \underline{c}_i); \overline{c}_i \leftarrow \min(\overline{C} - (tpf.\underline{P} - \min(c_i^{\min}, 0)), \overline{c}_i)$ 
23       $tpf \leftarrow tpf.next$ 
24  else // Step 3': task without fixed part ( $\overline{s}_i \geq \underline{e}_i$ ), adjust  $c_i, d_i$ 
25     $\overline{d}^* \leftarrow 0; s' \leftarrow \underline{s}_i$  // Track feasible start and duration
26    if  $\underline{d}_i \geq 1$  then // Initialize bounds over core  $[\underline{e}_i - 1, \overline{s}_i]$ 
27       $\underline{c}^* \leftarrow \underline{C} - tpf.prev.\overline{P} + \max(\overline{c}_i, 0); \overline{c}^* \leftarrow \overline{C} - tpf.prev.\underline{P} + \min(\underline{c}_i, 0)$ 
28      while  $tpf.time < \overline{s}_i$  do // Scan core interval  $[\underline{e}_i, \overline{s}_i]$ 
29         $\overline{d}^* \leftarrow \max(tpf.time - s', \overline{d}^*)$  // Record conflict-free span
30        if  $tpf.\underline{P} + \max(\underline{c}_i, 0) > \overline{C} \vee tpf.\overline{P} + \min(\overline{c}_i, 0) < \underline{C}$  then  $s' \leftarrow tpf.next.time$  // Reset start  $s'$ 
31        else checkIfMandatory( $i, tpf, \underline{C}, \overline{C}$ ) // Check mandatory condition (C3)
32        if  $\underline{d}_i \geq 1$  then
33           $\underline{c}^* \leftarrow \min(\underline{C} - tpf.\overline{P} + \max(\overline{c}_i, 0), \underline{c}^*); \overline{c}^* \leftarrow \max(\overline{C} - tpf.\underline{P} + \min(\underline{c}_i, 0), \overline{c}^*)$ 
34         $tpf \leftarrow tpf.next$ 
35      if  $\underline{d}_i \geq 1 \wedge tpf.time = \overline{s}_i$  then // Evaluate upper boundary point  $\overline{s}_i$ 
36         $\underline{c}^* \leftarrow \min(\underline{C} - tpf.\overline{P} + \max(\overline{c}_i, 0), \underline{c}^*); \overline{c}^* \leftarrow \max(\overline{C} - tpf.\underline{P} + \min(\underline{c}_i, 0), \overline{c}^*)$ 
37      if  $\underline{d}_i \geq 1$  then // Apply free-part consumption consistency (TC2)
38         $\underline{c}_i \leftarrow \max(\underline{c}^*, \underline{c}_i); \overline{c}_i \leftarrow \min(\overline{c}^*, \overline{c}_i)$ 
39       $\overline{d}^* \leftarrow \max(\overline{e}_i - s', \overline{d}^*); \underline{d}_i \leftarrow \min(\overline{d}^*, \underline{d}_i)$  // Apply duration consistency (TC3)

```

Algorithm 4: $\text{checkIfMandatory}(i, tp, \underline{C}, \overline{C})$

```

1 if  $tp.\#fp > 0 \wedge (tp.\underline{P} - \min(\underline{c}_i, 0) > \overline{C} \vee tp.\overline{P} - \max(\overline{c}_i, 0) < \underline{C})$  then
2    $p_i \leftarrow \text{true}$ 
3    $\overline{s}_i \leftarrow \min(tp.\text{time}, \overline{s}_i)$ 
4    $\underline{e}_i \leftarrow \max(tp.\text{next.time}, \underline{e}_i)$ 
5    $\text{deficit} \leftarrow \underline{C} - (tp.\overline{P} - \max(\overline{c}_i, 0))$ 
6    $\text{overload} \leftarrow \overline{C} - (tp.\underline{P} - \min(\underline{c}_i, 0))$ 
7   if  $\text{deficit} > 0$  then  $\underline{c}_i \leftarrow \max(\text{deficit}, \underline{c}_i)$ 
8   if  $\text{overload} < 0$  then  $\overline{c}_i \leftarrow \min(\text{overload}, \overline{c}_i)$ 

```

minimum capacity $\underline{C}$ to enforce. If all the consumption variables are fixed to a single value, the *Consumption* adjustment rule is not necessary. Similarly, the *Duration* adjustment rule is only needed if the tasks have variable durations. If these three rules can be ignored, only steps 1 and 2 need to be performed and the unnecessary loops at Lines 17 and 28 can be avoided.

Another optimization is to avoid considering some tasks when performing the timetabling algorithm. Indeed, fixed tasks that occur before the start of the earliest unfixed task or after the end of the latest unfixed task are not useful for filtering. Thus, such tasks do not need to be processed which reduces the number of time points in the profile. To do so, we use the same process as the *Fruitless Fixed Tasks Removal* of Gay et al. (2015a).

The following theorems establish the soundness, filtering consistency, and worst-case time complexity of Algorithm 3 (all proofs are deferred to Appendix A).

THEOREM 4 (SOUNDNESS OF ALGORITHM 3). *Algorithm 3 is sound: every domain reduction it performs removes only values that cannot appear in any solution of $\text{GeneralizedCumulative}(T, \underline{C}, \overline{C})$.*

THEOREM 5 (TIMETABLING FILTERING CONSISTENCY). *Upon termination without failure, Algorithm 3 achieves timetabling filtering consistency (Definition 7) with respect to the timeline $\mathcal{L}$ computed by Algorithm 2.*

Propagator Idempotency and Engine Fixpoint. While Theorem 5 proves timetabling filtering consistency with respect to the fixed timeline $\mathcal{L}$, Algorithm 3 is not idempotent because task domains are tightened in-place while $\mathcal{L}$ remains static. An immediate re-execution of Algorithm 3 on the narrowed domains could rebuild an updated timeline and achieve further pruning without external search branching. Algorithm 3 intentionally performs a single filtering pass over $\mathcal{L}$ to guarantee an $O(n^2)$ worst-case time bound per invocation. Global fixpoint consistency is achieved through the standard propagation queue of the CP solver engine, which automatically re-enqueues the propagator whenever variable bounds are modified.

THEOREM 6 (COMPLEXITY OF ALGORITHM 3). *Algorithm 3 has a worst-case time complexity of $O(n^2)$, where $n = |T|$.*

EXAMPLE 6. *When executing the timetabling algorithm on the same tasks as in Example 5, the adjustments are:*

- (1) *When processing task A at time point 1, its maximum consumption $\overline{c}_A$ is adjusted to 1 by the Consumption rule.*
- (2) *When processing task B at time point 2, the task is detected as conflicting at this time point by the Forbid rule. Its minimum start $\underline{s}_B$ is thus pushed to the next time point at time 3 where the adjustment stops as the task is no longer conflicting at this time. This also updates its minimum end to $\underline{e}_B = 6$ due to the internal relation $\underline{e}_B \geq \underline{s}_B + \underline{d}_B$.*
- (3) *When processing task C at time point 4, the task is detected as mandatory. The following adjustments are done:*

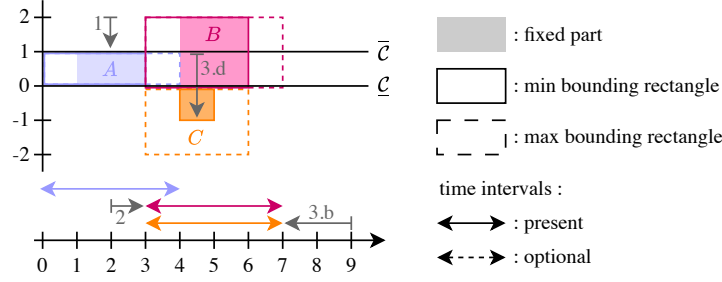


Fig. 5. Representation of the tasks of Example 5 after timetabling. Adjustments are shown as gray arrows when applicable.

- (a) The task is set to present.
 - (b) Its maximum starting time is set to $\bar{s}_C = 4$ which also updates its maximum ending time to $\bar{e}_C = 7$ due to the internal relation $\bar{e}_C \leq \bar{s}_C + \bar{d}_C$.
 - (c) Its minimum ending time is set to $\underline{e}_C = 5$.
 - (d) As the value $\text{overload} = -1$ is negative, the maximum consumption of the task is set to $\bar{c}_C = -1$.
- Note that even if task B is processed before task C and its fixed part increased, task C is adjusted based on the profile range which has been computed based on the state of task B at the start of the propagation.

The state of the tasks after timetabling is:

- A : $\langle s = [0, 1], d = [3, 4], e = [3, 4], c = 1, p = \text{true} \rangle$
- B : $\langle s = [3, 4], d = [3, 4], e = [6, 7], c = 2, p = \text{true} \rangle$
- C : $\langle s = [3, 4], d = [1, 3], e = [5, 7], c = [-2, -1], p = \text{true} \rangle$

Figure 5 shows the state of the tasks as well as the adjustments done.

4 Related Work

This section reviews existing approaches for modeling and solving producer-consumer constraints in constraint programming. We organize it around three main categories: foundational work on producer-consumer constraints, generalized multi-resource constraints, and closed-source commercial solvers.

4.1 Producer-Consumer constraints

The foundational work on producer-consumer problems in constraint-based scheduling was established by Cesta and Stella (1997) and extended by Laborie (2003). Laborie models resource constraints as tuples $\langle T, R, c, TE \rangle$, where c denotes the quantity of the resource R consumed (if $c < 0$) or produced (if $c > 0$) by activity T , and TE specifies the time extent during which the resource availability is affected. The proposed time extents include *FromStartToEnd*, *AfterStart*, *AfterEnd*, *BeforeStart*, *BeforeEnd*, and *Always*. Notice that these constructs are conceptually close to cumulative functions constructs on conditional time intervals, although they do not explicitly capture the conditional nature of the latter. The filtering descriptions in Cesta and Stella (1997) and Laborie (2003) are quite different from ours as they both rely on temporal networks and precedence graphs between so-called production and consumption events, rather than on the profile-based reasoning over mandatory parts used in this paper. Nevertheless, the filtering looks similar in spirit to the timetabling algorithm we present. For instance, Cesta and Stella (1997) describes an optimistic resource profile and a pessimistic resource profile, which are similar to our minimum and maximum profiles. The proposed algorithm achieves a worst-case time complexity of $O(n^2)$,

Feature / Aspect	Beldiceanu and Carlsson (2002)	This Work
Data structure	Event array sweep	Doubly-linked <i>Profile</i> with $O(1)$ boundary pointers
Backward adjustment	Single adjustment per propagation call	Chained adjustments in a single pass (Ex. 7)
Consumption pruning	Restricted to single-event span	Minimum overlapping interval $[\underline{e}_i - 1, \bar{s}_i]$ (Ex. 8)
Duration pruning	Measured from single conflicting point	Longest conflict-free sub-interval (Ex. 9)
Optional tasks	Indirect via dummy resource	Native conditional intervals with direct presence pruning

Table 2. Key differences between the timetabling algorithm of Beldiceanu and Carlsson (2002) and the profile-based algorithm proposed in this work.

where n is the number of tasks. However, it does not provide implementation details, such as the profile data structures. The work of Schutt and Stuckey (2016) builds on these ideas to provide explanations suitable to be used by lazy clause generation solvers.

4.2 Generalized multi-resource constraint

Beldiceanu and Carlsson (2002) introduced a timetabling filtering algorithm for the GeneralizedCumulative constraint with k alternative resources. This constraint does not rely on conditional task intervals or cumulative functions (which were only introduced later by Laborie, Rogerie, Shaw, et al. (2009)); instead, it operates directly on arrays of integer variables. The signature of the constraint is:

$$\text{cumulatives}(s[1, n], d[1, n], e[1, n], c[1, n], m[1, n], \underline{C}[1, k], \overline{C}[1, k])$$

where s , d , e , and c denote respectively the start times, durations, end times, and consumptions (which can be positive or negative). The variables m indicate, for each task, the resource on which it executes, with domain on $\{1, \dots, k\}$. Finally, $[\underline{C}[j], \overline{C}[j]]$ specifies the capacity range of resource j .

Although this constraint does not directly support conditional task intervals, it is straightforward to connect the two modeling approaches. Indeed, a dummy resource can be used to indicate if a task is not present in the multi-resource formulation. Conversely, the conditional task approach supports multiple resources by using an additional *alternative* constraint (Laborie and Rogerie 2008) that represent the choice of alternative resources. This requires duplicating the activity once for each alternative resource, but it has the advantage that each alternative task can be filtered independently during propagation. In contrast, the multi-resource formulation propagation can only exploit the minimum over the different possible starts across all resources, which can lead to a weaker filtering.

The filtering algorithm of Beldiceanu and Carlsson (2002) proceeds by collecting a set of tasks at relevant time points before performing propagation. This algorithm is implemented in the open-source solver Gecode and according to its documentation, the same algorithm is also implemented in SICStus (Carlsson et al. 2025). The algorithm presents several substantial differences with our algorithm, summarized in Table 2.

First, our approach builds on the profile-based paradigm of Gay et al. (2015a) and Gingras and C. Quimper (2016) for the classical cumulative constraint, which constructs an explicit resource profile and tests each task against it. We rely on the doubly-linked *Profile* data structure of Gingras and C. Quimper (2016) to prune task bounds both forward and backward, retrieving the starting rectangle in $O(1)$ via direct pointers. In contrast to Gay et al. (2015a), which represents the profile as an unsorted list of rectangles, direct boundary pointers eliminate search overhead. Consequently, although the worst-case quadratic time complexity per call remains $O(n^2)$ as in Beldiceanu and Carlsson (2002), our algorithm performs substantially fewer fixpoint iterations and avoids

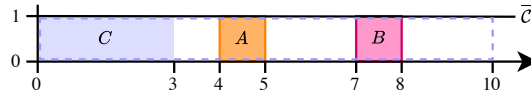


Fig. 6. Example 7: three tasks before timetabling with $\bar{C} = 1$. Tasks A ([4, 5), consumption 1) and B ([7, 8), consumption 1) are fixed and present; the dashed blue rectangle shows the time window $[s_C, \bar{e}_C) = [0, 10)$ of task C (fixed duration 3, consumption 1).

redundant passes. Second, as demonstrated in the following examples, our algorithm prunes the consumption and duration attributes of tasks more effectively than Beldiceanu and Carlsson (2002).

Backward Propagation Difference. For the adjustment of the maximum end time ($\bar{e}$) of tasks due to the *Forbid* rule, when a task requires two or more consecutive adjustments, our algorithm performs all the adjustments in a single call to the timetabling algorithm compared to the version of Beldiceanu and Carlsson (2002) where only one adjustment is done per call. Indeed, in Beldiceanu and Carlsson (2002), detection and adjustment are performed only in a forward fashion. Thus, the filtering of the maximum end time of tasks is done only in regards to the current end of the task time window. This means that if a task has its maximum end time adjusted, whether or not the maximum end time can be further adjusted will be considered in the next call to the filtering algorithm. In practice, this may lead to situations where the filtering algorithm of Beldiceanu and Carlsson (2002) needs several consecutive calls in order to completely adjust the end time of a task and reach a fixed point. In contrast the algorithm presented in this paper does such adjustments in a single pass as illustrated in Example 7.

EXAMPLE 7. Let us consider three tasks:

- Task A is fixed and present, starts at 4, ends at 5 and has a consumption of 1.
- Task B is fixed and present, starts at 7, ends at 8 and has a consumption of 1.
- Task C is present, has a minimum start of 0, a maximum end of 10, a fixed duration of 3 and a consumption of 1.

The maximum capacity of the resource $\bar{C}$ is 1. Figure 6 shows the three tasks before propagation. The dashed blue rectangle represents the time window $[s, \bar{e})$ of task C.

During the backward propagation of our timetabling algorithm, task C is detected as infeasible at times 7 then 4, and its maximum end time is successively updated to 7 then 4 in a single call. The propagation from Beldiceanu and Carlsson (2002) iterates the profile in one direction and detects the infeasibility of task C only at time 7 and adjusts its maximum end at 7 during the first call. A second call to the algorithm is necessary in order to adjust the maximum end at time 4.

Consumption Adjustment Difference. For consumption adjustment, the difference occurs in the case where a task has no fixed part (second case of the *Consumption* filtering rule). In this case, our algorithm considers the maximum consumption available over the minimum overlapping interval of the task (Gay et al. 2015b).

The propagation from Beldiceanu and Carlsson (2002) uses another consumption adjustment rule (see Alg. 4 in their paper). It considers the maximum available consumption only if the task examined has both its earliest completion time ($\underline{e}$) and its latest start time ($\bar{s}$) in the same event interval. That means that if one or more other tasks affect the available consumption between the $\underline{e}$ and $\bar{s}$ of a task, leading them to occur during different events, the consumption of the task is not adjusted. Example 8 illustrates this.

EXAMPLE 8. Let us consider three tasks:

- Task A is fixed and present, starts at 4, ends at 12 and has a consumption of 2.

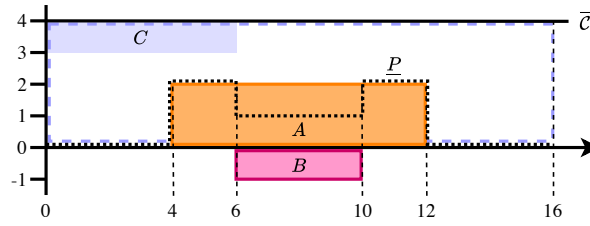


Fig. 7. Example 8: three tasks before timetabling with $\bar{C} = 4$. Tasks A ([4, 12], consumption 2) and B ([6, 10], consumption -1) are fixed; the dashed blue rectangle shows task C's maximum bounding rectangle (time window [0, 16], fixed duration 6, consumption $\in [1, 4]$). The dotted black line is the minimum profile $\underline{P}$.

- Task B is fixed and present, starts at 6, ends at 10 and has a consumption of -1.
- Task C has a minimum start of 0, a maximum end of 16 and a fixed duration of 6. Its consumption is in the interval [1, 4]

The maximum capacity of the resource $\bar{C}$ is 4. Figure 7 shows the three tasks before propagation. The dashed blue rectangle represents task C maximum bounding rectangle. The dotted black line shows the minimum profile $\underline{P}$.

During the execution of the timetabling algorithm, the maximum consumption of task C is adjusted to 3 as this value corresponds to the maximum available consumption over its minimum overlapping interval (from time 5 to time 10). This filtering does not occur with the GeneralizedCumulative constraint from Beldiceanu and Carlsson (2002) as the e_C and s_C of task C are not in the same event due to the presence of task B. If the task B is removed, then both algorithms obtain the same filtering as the consumption of task C is adjusted to 2.

Duration Adjustment Difference. The difference in filtering on the maximum duration adjustment of the tasks occurs if the profile prevents a task from being present in at least two different parts of its time window. In this case, the algorithm presented in this paper identifies the longest interval in the profile where the task can fit and adjusts the task's maximum duration accordingly.

In contrast, the filtering of Beldiceanu and Carlsson (2002) considers one single conflict point at a time, rather than the entire conflicting interval. When encountering conflicting time point in the profile, the task maximum duration is updated based on the maximum span between the conflicting time point and either the minimum start or the maximum end of the task. Example 9 illustrates this.

EXAMPLE 9. Let us consider three tasks:

- Task A is fixed and present, starts at 3, ends at 6 and has a consumption of 3.
- Task B is fixed and present, starts at 10, ends at 14 and has a consumption of 3.
- Task C has a minimum start of 0, a maximum end of 16 and a consumption of 2. Its duration is in the interval [2, 16]

The maximum capacity of the resource $\bar{C}$ is 4. Figure 8 shows the three tasks before propagation. The dashed blue rectangle represents task C maximum bounding rectangle. The black dotted line shows the maximum capacity ($\bar{C}$). In this example, both tasks A and B prevent task C from being present at the same time.

During the execution of the timetabling algorithm, the maximum duration of task C is adjusted to 4, as this is the span of the longest feasible placement interval (from time 6 to time 10). When filtering with the propagation algorithm from Beldiceanu and Carlsson (2002), the duration is instead adjusted to 10, since this corresponds to both the span available after task A and the span available before task B.

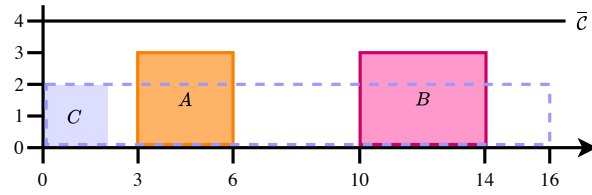


Fig. 8. Example 9: three tasks before timetabling with $\bar{C} = 4$. Tasks A ([3, 6), consumption 3) and B ([10, 14), consumption 3) are fixed; the dashed blue rectangle shows task C's maximum bounding rectangle (time window [0, 16), consumption 2, duration $\in [2, 16]$). The dotted black line shows the maximum capacity $\bar{C}$.

4.3 Closed source solvers

Both CPOptimizer and OptalCP natively implement the cumulative functions modeling paradigm. While the conceptual foundations of conditional intervals and cumulative function algebra have been extensively documented by Laborie and Rogerie (2008), Laborie, Rogerie, Shaw, et al. (2009), and Laborie, Rogerie, Shaw, et al. (2018), both engines are closed-source commercial software. Consequently, their concrete algorithmic implementations, internal profile data structures, and propagation mechanics for range constraints on cumulative functions have not been published. Based on the filtering and the number of backtracks on some examples, we observed that CPOptimizer achieves a filtering similar to the Gecode implementation when a cumulative function requires a generalized cumulative constraint. For specific cases, like when a cumulative function models a non-overlap constraint, the constraint is reformulated by CPOptimizer into a standard disjunctive constraint for which strong dedicated filtering exists (such as edge-finding, not-first/not-last, etc.).

Contrary to ours, the CPOptimizer modeling API does not allow cumulative functions to take negative values. Indeed, while cumulative tasks can have a negative contribution, a cumulative function whose value becomes negative will cause a failure in CPOptimizer. This is equivalent to having a constraint $f \geq 0$ on any cumulative function f used in the model. It is not known whether this limitation is due to a design choice or a limitation in the filtering algorithm used for the constraint.

5 Experiments

Our approach and filtering algorithm are open source and available in MaxiCP, a Java solver that extends MiniCP (Michel et al. 2021). We compare it with the closed-source state-of-the-art commercial solver CPOptimizer as well as the open-source solver Gecode that implements a version of the GeneralizedCumulative constraint of Beldiceanu and Carlsson (2002). As cumulative functions are not available in Gecode, we carefully modeled them in a way similar to how they would be flattened by Algorithm 1. Three cumulative problems that involve reservoirs or negative consumptions are considered. We use the same fixed static search strategy for all solvers and problems to ensure that only the filtering strength and propagation speed influence the results. Although OptalCP also supports cumulative functions, it does not allow fixing the search strategy as CPOptimizer does. Since this paper focuses on filtering and modeling rather than overall solving performance, including a solver with a free search would conflate search heuristics with propagation strength, making it impossible to isolate the contribution of the filtering algorithm; we therefore exclude it from the comparison. The experiments were carried out on a MacBook Pro M3 with 32 GB of memory. The source code of the models is available at (Pierre Schaus and Charles Thomas and Roger Kameugne 2026).

For each problem, we report results using a cactus plot showing the cumulative number of instances solved as a function of time. We also provide a pairwise comparison of solvers in terms of the number of backtracks on instances solved by both solvers.

Each model involves a set of tasks $i \in T$. Recall that the attributes for each task are s_i the start time, d_i the duration, e_i the end time, c_i the consumption, p_i the execution status (present or absent) when applicable.

5.1 The RCPSP with Consumption and Production of Resources (RCPSP-CPR)

The Resource-Constrained Project Scheduling Problem with Consumption and Production of Resources (RCPSP-CPR) (Koné et al. 2013) is an extension of the classic Resource-Constrained Project Scheduling Problem (RCPSP) (Dike 1964; Ding et al. 2023) with additional reservoir resources that must always be kept positive. Each task is present and, in addition to classical renewable resource consumption (modeled with pulse functions), tasks also consume a specified amount of reservoir resources at their start and produce a specified amount at their end (modeled with stepAtStart and stepAtEnd). The objective is to minimize the makespan.

Model. T denotes the set of tasks, Rn is the set of renewable resources, Rs is the set of reservoir resources, and P is the set of precedences between pairs of tasks. The capacity of each renewable resource $k \in Rn$ is denoted C_k and the initial level of each reservoir resource $u \in Rs$ is denoted C_u^r . Each task i consumes c_{ki} units of the renewable resource $k \in Rn$, consumes c_{ui}^- units of reservoir resource $u \in Rs$ at its start time and produces c_{ui}^+ units of reservoir resource $u \in Rs$ at its end time.

The model is written as:

$$\text{minimize } \max_{i \in T} e_i \quad (12)$$

subject to

$$e_i \leq s_j \quad \forall (i, j) \in P \quad (13)$$

$$ren_k = \sum_{i \in T} pulse(i, c_{ki}) \quad \forall k \in Rn \quad (14)$$

$$res_u = step(0, C_u^r) - \sum_{i \in T} stepAtStart(i, c_{ui}^-) + \sum_{i \in T} stepAtEnd(i, c_{ui}^+) \quad \forall u \in Rs \quad (15)$$

$$alwaysIn(ren_k, -\infty, C_k) \quad \forall k \in Rn \quad (16)$$

$$alwaysIn(res_u, 0, +\infty) \quad \forall u \in Rs \quad (17)$$

The objective (12) is to minimize the makespan. The constraints (13) ensure the precedences between tasks. The renewable and reservoir resources are constrained in (14) and (15) respectively. The resource constraints are posted at (16) and (17) respectively.

The search consists in a static binary branching that selects variables in the order in which tasks are declared in the instance file. The first un-assigned start variable is selected and assigned to its minimum value in the left branch. The right branch removes the minimum value from the domain.

Results. We experiment on 55 instances with 10 tasks adapted from Koné et al. (2013). A time limit of 600 s is set per instance for finding the first feasible solution. As shown in Figure 9, MaxiCP solves more instances than Gecode and roughly the same set of instances as CPOptimizer.

Figure 10 reports for each pair of solvers, for each instance solved commonly by both solvers, an x, y -plot of the number of backtracks. The comparison between CPOptimizer and MaxiCP reveals that the number of backtracks is nearly identical, suggesting that CPOptimizer is simply faster on this problem, as both solvers achieve similar levels of filtering on this problem, but MaxiCP requires fewer backtracks than Gecode on some of the instances.

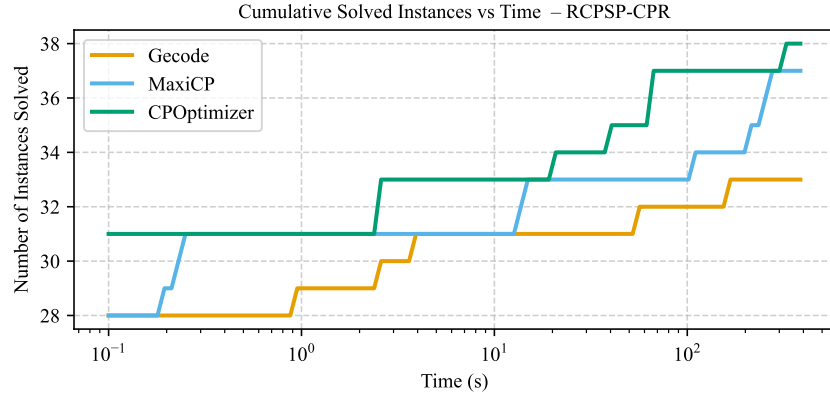


Fig. 9. Number of solved instances as a function of time for the RCPSP-CPR Problem.

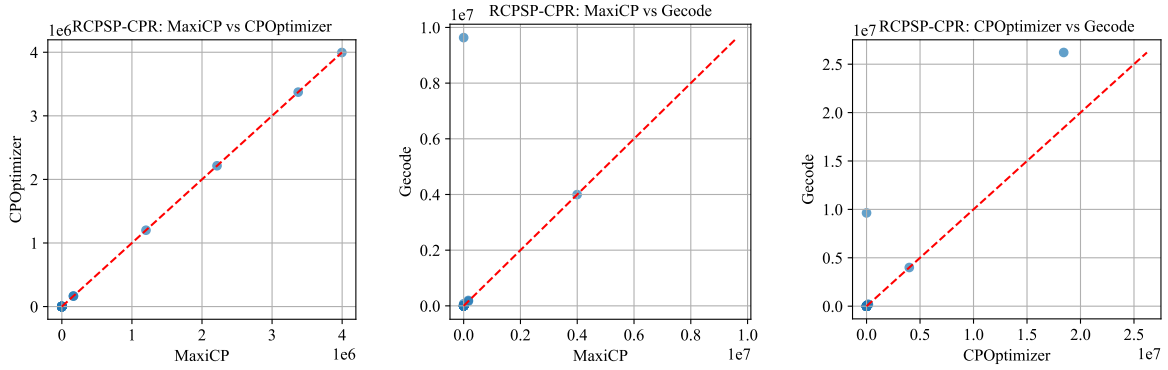


Fig. 10. Failure comparison for the RCPSP-CPR

5.2 The Single Machine with Inventory Constraints (SMIC)

The Single Machine with Inventory Constraints (SMIC) problem was introduced by [Davari et al. \(2020\)](#). Each task has a release date $release_i$, a fixed duration, and a positive or negative inventory consumption $invent_i$ that occurs at the start of the activity (modeled with `stepAtStart`). The inventory starts at a given value $initInventory$ and must stay within a specified range $([0, capInventory])$. Activities cannot overlap in time. The objective is to minimize the makespan.

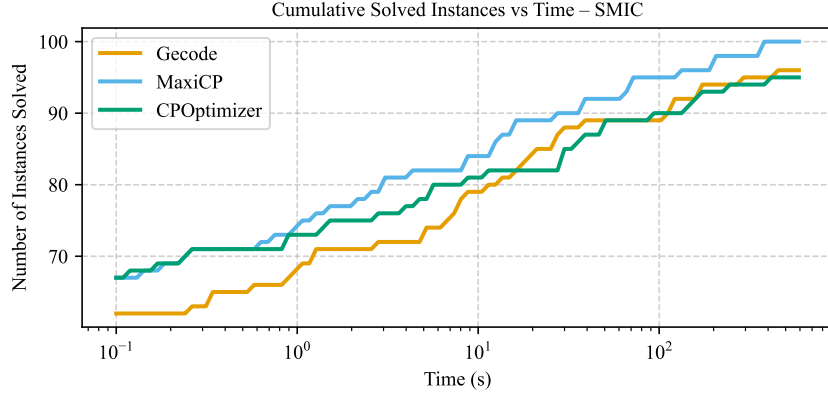


Fig. 11. Number of solved instances as a function of time for the SMIC Problem

Model. The model is written as:

$$\text{minimize } \max_{i \in T} e_i \quad (18)$$

subject to

$$s_i \geq \text{release}_i, \quad \forall i \in T \quad (19)$$

$$\text{res} = \text{step}(0, \text{initInventory}) + \sum_{i \in T} \text{stepAtStart}(i, \text{invent}_i) \quad (20)$$

$$\text{alwaysIn}(\text{res}, 0, \text{capaInventory}) \quad (21)$$

$$\text{noOverlap} = \sum_{i \in T} \text{pulse}(i, 1) \quad (22)$$

$$\text{alwaysIn}(\text{noOverlap}, -\infty, 1) \quad (23)$$

The objective (18) is to minimize the makespan (i.e., the time at which all tasks are completed). The release dates are ensured by the relation (19). The reservoir resource is constrained in relations (20) and (21) to ensure that the capacity of the inventory is not violated. The noOverlap constraint is enforced with the relations (22) and (23) to ensure the no-overlapping of activities. Notice that those are modeled using cumulative functions rather than with global dedicated non-overlap constraints on task intervals to ensure that the results are solely impacted by the filtering of the cumulative constraint rather than by the possibly different filtering of the non-overlap constraint. This, unfortunately, did not appear possible for CPOptimizer which recognizes that the cumulative function is used to model a non-overlap constraint and automatically reformulates it using a non-overlap constraint with stronger filtering. To avoid this bias, we use the timetabling filtering algorithm adapted from this paper for CPOptimizer, to ensure consistent comparison with other solvers.

The search consists in a static branching heuristic that selects variables in the order in which tasks are declared in the instance file. The first un-assigned start variable is selected and assigned to its minimum value in the left branch. The right branch removes the minimum value from the domain.

Results. We experiment on instances with 10 and 20 tasks from Davari et al. (2020). A time limit of 600 s is set per instance for finding and proving optimality. As shown in Figure 11, CPOptimizer solves fewer instances while MaxiCP and Gecode roughly have the same behavior on this problem.

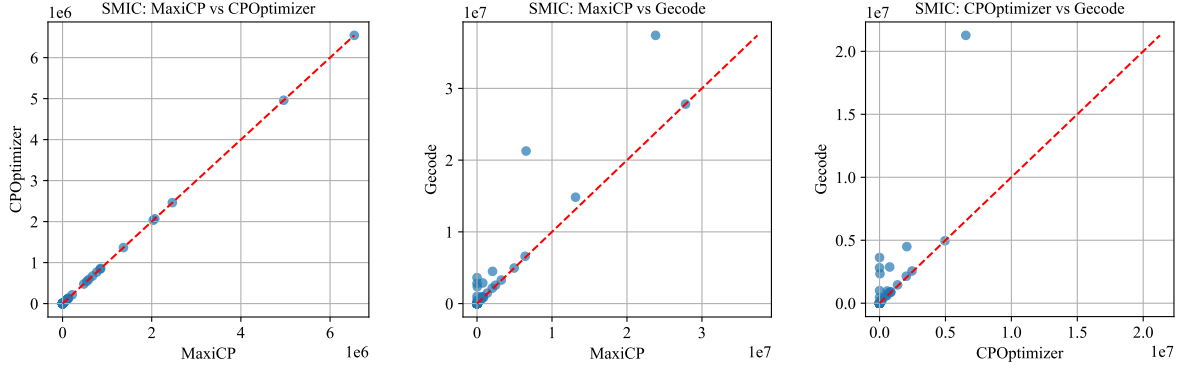


Fig. 12. Failure comparison for the SMIC

Figure 12 reports for each pair of solvers, for each instance solved commonly by both solvers, an x, y -plot of the number of backtracks. This analysis shows that CPOptimizer and MaxiCP require the same number of backtracks on this problem, confirming comparable filtering strength between the two solvers. Gecode, in contrast, misses some of the adjustments discussed in the sub-section 4.2, resulting in weaker pruning; nonetheless, the number of instances it solves remains close to that of CPOptimizer.

5.3 The Maximum Energy Scheduling Problem (MESP)

The Maximum Energy Scheduling Problem (MESP) consists in scheduling optional tasks with variable durations and resource demands, which can be positive or negative, on a single resource with fixed capacity $\bar{C}$. The energy of a task is the product of its duration and its demand. The objective is to maximize the total positive energy consumption of the resource while ensuring that the resource capacity is not exceeded at any time. All filtering rules are beneficial for this problem, making it the most relevant benchmark for evaluating the proposed filtering algorithm.

Model. The model is written as:

$$\text{maximize} \quad \sum_{i \in T | p_i = \text{true}} \max(c_i, 0) \cdot d_i \quad (24)$$

subject to

$$\text{resource} = \sum_{i \in T} \text{pulse}(i, c_i) \quad (25)$$

$$\text{alwaysIn}(\text{resource}, -\infty, \bar{C}) \quad (26)$$

The objective (24) is to maximize the sum of the positive energies of the tasks that are present. The constraint (25) sets up the reservoir resource and its capacity is constrained by (26). While MESP is formulated as an optimization problem in (24), our scalability experiment evaluates the performance of the filtering algorithm when driven by a greedy constructive heuristic that seeks to maximize energy in a single dive without backtracking, stopping at the first feasible solution. This isolates propagation throughput on massive task sets from search exploration.

Note that the resource demand variables of tasks c_i may have negative values in their domain in this problem. As explained in *Filtering Differences* section, CPOptimizer does not allow cumulative functions to have a negative value. In order to accommodate this restriction, the model for CPOptimizer has been adapted in the following

way: The cumulative function *resource* is shifted up in height at the time 0 by a step function of height $neg^* = \sum_{i \in T} -\min(c_i, 0)$ which corresponds to the sum of all maximum negative values of the tasks' resource demands. The capacity $\bar{C}$ of the resource is also shifted by the same value. In formal terms, constraint (25) is changed to: $resource = step(0, neg^*) + \sum_{i \in T} pulse(i, c_i)$ and constraint (26) is changed to $alwaysIn(resource, -\infty, \bar{C} + neg^*)$.

This ensures that the cumul function *resource* remains non-negative, as required by CPOptimizer.

The search heuristic selects tasks in the fixed order of their declaration in the instance file. For each task, variables are fixed greedily to their maximum domain values in the following order: presence $p_i \leftarrow true$, demand $c_i \leftarrow \bar{c}_i$, duration $d_i \leftarrow \bar{d}_i$, and end time $e_i \leftarrow \bar{e}_i$. Two branches are created: the left branch fixes the value while the right branch removes it from the domain. The search stops at the first solution found.

Instance Generation. We generated 60 MESP instances with the total number of tasks n ranging from 6 to 12,800. For each specified size, five distinct instances were generated using different random seeds. In every instance, the task set is divided into tasks drawing resource capacity (80% of the total, i.e. $n - \lfloor n/5 \rfloor$ tasks) and tasks replenishing capacity (20%, i.e. $\lfloor n/5 \rfloor$ tasks). All instances share a fixed resource capacity of $\bar{C} = 10$ and a maximum task duration of $L_{\max} = 20$. The scheduling horizon H is set to $H = \max(50, 5n)$. For tasks drawing resource capacity, the maximum resource demand $\bar{c}_i$ is drawn uniformly from $[1, 10]$, and the minimum demand $\underline{c}_i$ from $[1, \bar{c}_i]$. For tasks replenishing capacity, the maximum resource demand $\bar{c}_i$ is drawn from $[-10, -1]$ and the minimum demand $\underline{c}_i$ from $[\bar{c}_i, -1]$. The minimum start time of every task is drawn uniformly from $[0, H - L_{\max} - 1]$, ensuring no task is forced to extend beyond the horizon.

Results. A time limit of 2,000 s and a memory limit of 16 GB is set per instance for finding a feasible solution. Given the large number of tasks in some instances, this experiment is designed to test the scalability of the filtering. We chose MESP for this scalability experiment because, unlike RCPSP-CPR and SMIC, its benchmark instances span a much wider range of sizes, with some instances containing several thousand tasks and resource requirements far exceeding those found in the other two problems. This large variability in instance size makes MESP particularly well-suited to stress-test the filtering algorithm under realistic, large-scale conditions, and to observe how its runtime and pruning behavior evolve as the problem size grows. In contrast, RCPSP-CPR and SMIC instances remain comparatively small and homogeneous in size, and would not allow us to draw meaningful conclusions about scalability. As can be observed in Figure 13, our approach performs very well with more instances solved. Both Gecode and CPOptimizer fail to solve the largest instances due to timeouts and memory limits respectively.

Figure 14 reports for each pair of solvers, for each instance solved commonly by both solvers, an x, y -plot of the number of backtracks. As can be seen, our approach avoids backtracking, unlike the other solvers. Interestingly, CPOptimizer requires fewer backtracks than Gecode although it solves fewer instances.

5.4 Why not compare with solvers using MiniZinc (Mears et al. 2014)?

The MiniZinc modeling language provides a cumulatives constraint that allows for negative consumption. Unfortunately, with the exception of SICStus and Gecode, this constraint is decomposed in other solvers. Also the MiniZinc API for this constraint does not support explicit access to the *end* variables. Being able to access the *end* variables directly rather than relying on the relation $start + duration = end$ is crucial for the problems considered, as the *end* variables can be constrained independently, as explained in the next example.

EXAMPLE 10. Assume a task with the following properties: $s = [3, 7]$, $d = [1, 5]$, $e = [8, 8]$. The mandatory part of this task is $[\bar{s}, \underline{e} - 1] = [7, 7]$. If the end variable is not available, the mandatory part inferred from s and d is $[\bar{s}, \underline{s} + \underline{d} - 1] = \emptyset$.

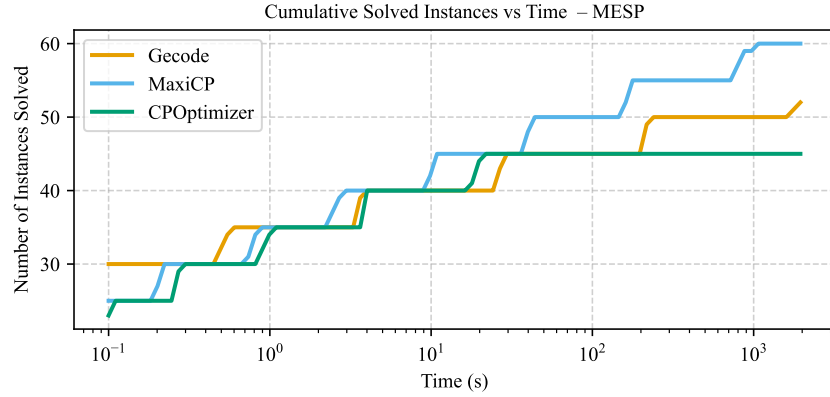


Fig. 13. Number of solved instances as a function of time for the MESP Problem.

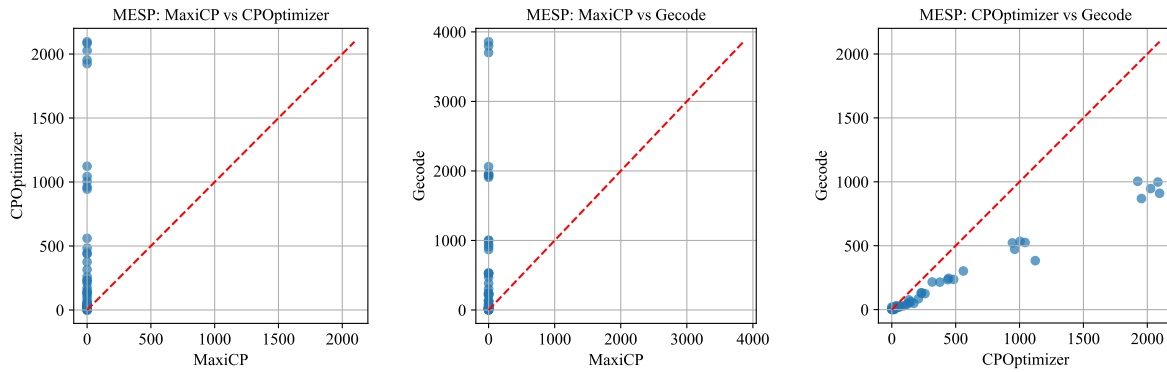


Fig. 14. Failure comparison for the MESP

As a result, the propagation when using MiniZinc for these problems can be much weaker, making the comparison unfair. This was confirmed experimentally and can be verified with MiniZinc models.

6 Conclusion

We presented an implementation of the cumulative functions modeling paradigm for scheduling problems. Our implementation applies a flattening procedure to cumulative function expressions, producing a set of cumulative tasks with positive or negative resource consumption, which are then passed to a `GeneralizedCumulative` constraint.

We introduced a novel timetabling filtering algorithm for the `GeneralizedCumulative` constraint that operates on conditional time-interval variables and supports negative resource consumption. This algorithm is simpler and achieves stronger pruning than the one proposed by [Beldiceanu and Carlsson \(2002\)](#).

Experimental results demonstrate that the proposed approach is scalable and competitive with other solvers on three cumulative scheduling problems with diverse characteristics, including optional tasks and negative resource consumption.

As future work, we plan to extend the filtering algorithm to generalize the Overload/Underload checking and edge-finding rules as in (Kameugne et al. 2023, 2024; P. Ouellet and C. Quimper 2013) in the context of producers and consumers. We also intend to explore explanations for the generalized cumulative constraint, similar to those proposed by Schutt, Feydy, et al. (2011), and to investigate whether a CP-SAT solver such as OR-Tools (Perron et al. 2023) could benefit from them.

References

- A. Aggoun and N. Beldiceanu. 1993. “Extending CHIP in order to solve complex scheduling and placement problems.” *Mathematical and computer modelling*, 17, 7, 57–73.
- P. Baptiste, C. Pape, and W. Nuijten. 2012. *Constraint-Based Scheduling: Applying Constraint Programming to Scheduling Problems*. International Series in Operations Research & Management Science. Springer US. ISBN: 9781461514794. <https://books.google.cm/books?id=qUzhBwAAQBAJ>.
- N. Beldiceanu and M. Carlsson. 2002. “A New Multi-resource cumulatives Constraint with Negative Heights.” In: *CP (Lecture Notes in Computer Science)*. Vol. 2470. Springer, 63–79.
- N. Beldiceanu, M. Carlsson, S. Demasse, and T. Petit. 2007. “Global constraint catalogue: Past, present and future.” *Constraints*, 12, 1, 21–62.
- Q. Cappare and P. Schaus. 2017. “Rescheduling railway traffic on real time situations using time-interval variables.” In: *International Conference on the Integration of Constraint Programming, Artificial Intelligence, and Operations Research (CPAIOR)*. Springer, 312–327.
- Q. Cappare, C. Thomas, P. Schaus, and L.-M. Rousseau. 2018. “A constraint programming approach for solving patient transportation problems.” In: *International Conference on Principles and Practice of Constraint Programming (CP)*. Springer, 490–506.
- M. Carlsson et al.. June 2025. *SICStus Prolog User’s Manual*. (Release 4.10.1 ed.). Version 4.10.1. RISE Research Institutes of Sweden AB. Kista, Sweden, (June 2025). <https://sicstus.sics.se/sicstus/docs/latest4/pdf/sicstus.pdf>.
- A. Cesta and C. Stella. 1997. “A time and resource problem for planning architectures.” In: *European Conference on Planning*. Springer, 117–129.
- M. Davari, M. Ranjbar, P. De Causmaecker, and R. Leus. 2020. “Minimizing makespan on a single machine with release dates and inventory constraints.” *European Journal of Operational Research*, 286, 1, 115–128.
- S. H. Dike. 1964. “Project scheduling with resource constraints.” *IEEE Transactions on Engineering Management*, EM-11, 4, 155–157. doi:10.1109/TEM.1964.6446423.
- H. Ding, C. Zhuang, and J. Liu. 2023. “Extensions of the resource-constrained project scheduling problem.” *Automation in Construction*, 153, 104958. doi:<https://doi.org/10.1016/j.autcon.2023.104958>.
- M. R. Garey and D. S. Johnson. 1979. *Computers and Intractability: A Guide to the Theory of NP-Completeness*. W. H. Freeman.
- S. Gay, R. Hartert, and P. Schaus. 2015a. “Simple and Scalable Time-Table Filtering for the Cumulative Constraint.” In: *CP (Lecture Notes in Computer Science)*. Vol. 9255. Springer, 149–157.
- S. Gay, R. Hartert, and P. Schaus. 2015b. “Time-table disjunctive reasoning for the cumulative constraint.” In: *International Conference on the Integration of Constraint Programming, Artificial Intelligence, and Operations Research (CPAIOR)*. Springer, 157–172.
- R. Gedik, D. Kalathia, G. Egilmez, and E. Kirac. 2018. “A constraint programming approach for solving unrelated parallel machine scheduling problem.” *Computers & Industrial Engineering*, 121, 139–149.
- V. Gingras and C. Quimper. 2016. “Generalizing the Edge-Finder Rule for the Cumulative Constraint.” In: *IJCAI. IJCAI/AAAI Press*, 3103–3109.
- R. Kameugne, S. B. Fetgo, T. Noulamo, and C. T. Djamegni. 2023. “Horizontally Elastic Edge Finder Rule for Cumulative Constraint Based on Slack and Density.” In: *CP (LIPIcs)*. Vol. 280. Schloss Dagstuhl - Leibniz-Zentrum für Informatik, 20:1–20:17.
- R. Kameugne, S. B. Fetgo, T. Noulamo, and C. T. Djamegni. 2024. “Improved timetable edge finder rule for cumulative constraint with profile.” *Comput. Oper. Res.*, 172, 106795.
- J. Kinable, T. Wauters, and G. V. Berghe. 2014. “The concrete delivery problem.” *Computers & Operations Research*, 48, 53–68.
- D. Kizilay, D. T. Eliyi, and P. Van Hentenryck. 2018. “Constraint and mathematical programming models for integrated port container terminal operations.” In: *International Conference on the Integration of Constraint Programming, Artificial Intelligence, and Operations Research (CPAIOR)*. Springer, 344–360.
- O. Koné, C. Artigues, P. Lopez, and M. Mongeau. 2013. “Comparison of mixed integer linear programming models for the resource-constrained project scheduling problem with consumption and production of resources.” *Flexible Services and Manufacturing Journal*, 25, 25–47.
- R. Kumar, G. Sen, S. Kar, and M. K. Tiwari. 2018. “Station dispatching problem for a large terminal: A constraint programming approach.” *Interfaces*, 48, 6, 510–528.
- P. Laborie. 2003. “Algorithms for propagating resource constraints in AI planning and scheduling: Existing approaches and new results.” *Artificial Intelligence*, 143, 2, 151–188.

- P. Laborie and J. Rogerie. 2008. “Reasoning with Conditional Time-Intervals.” In: *FLAIRS*. AAAI Press, 555–560.
- P. Laborie, J. Rogerie, P. Shaw, and P. Vilim. 2009. “Reasoning with Conditional Time-Intervals. Part II: An Algebraical Model for Resources.” In: *FLAIRS*. AAAI Press.
- P. Laborie, J. Rogerie, P. Shaw, and P. Vilim. 2018. “IBM ILOG CP optimizer for scheduling: 20+ years of scheduling with constraints at IBM/ILOG.” *Constraints*, 23, 210–250.
- C. Liu, D. M. Aleman, and J. C. Beck. 2018. “Modelling and solving the senior transportation problem.” In: *International Conference on the Integration of Constraint Programming, Artificial Intelligence, and Operations Research (CPAIOR)*. Springer, 412–428.
- C. Mears, A. Schutt, P. J. Stuckey, G. Tack, K. Marriott, and M. Wallace. 2014. “Modelling with option types in MiniZinc.” In: *International Conference on the Integration of Constraint Programming, Artificial Intelligence, and Operations Research (CPAIOR)*. Springer, 88–103.
- L. Michel, P. Schaus, and P. Van Hentenryck. 2021. “MiniCP: a lightweight solver for constraint programming.” *Mathematical Programming Computation*, 13, 1, 133–184. ISBN: 1867-2957. doi:[10.1007/s12532-020-00190-7](https://doi.org/10.1007/s12532-020-00190-7).
- P. Ouellet and C. Quimper. 2013. “Time-Table Extended-Edge-Finding for the Cumulative Constraint.” In: *CP (Lecture Notes in Computer Science)*. Vol. 8124. Springer, 562–577.
- Y. Ouellet and C.-G. Quimper. 2019. “Processing times filtering for the Cumulative constraint.” In: *Doctoral Program of the International Conference on Principles and Practice of Constraint Programming (CP)*.
- L. Perron, F. Didier, and S. Gay. 2023. “The CP-SAT-LP Solver.” In: *International Conference on Principles and Practice of Constraint Programming (CP) (Leibniz International Proceedings in Informatics (LIPIcs))*. Ed. by R. H. C. Yap. Vol. 280. Schloss Dagstuhl – Leibniz-Zentrum für Informatik, Dagstuhl, Germany, 3:1–3:2. ISBN: 978-3-95977-300-3. doi:[10.4230/LIPIcs.CP.2023.3](https://doi.org/10.4230/LIPIcs.CP.2023.3).
- G. Pesant. 2004. “A regular language membership constraint for finite sequences of variables.” In: *International conference on principles and practice of constraint programming*. Springer, 482–495.
- Pierre Schaus and Charles Thomas and Roger Kameugne. 2026. *GeneralizedCumulative: Source code repository*. <https://github.com/aia-uclouvain/GeneralizedCumulative>. Accessed: 2026-04-24. (2026).
- J.-C. Régim. 1994. “A filtering algorithm for constraints of difference in CSPs.” In: *AAAI*. Vol. 94, 362–367.
- P. Schaus, G. Derval, A. Delecluse, L. Michel, and P. V. Hentenryck. 2026. *MaxiCP: A Not So Mini Constraint Programming Solver*. <http://www.maxicp.org/>. (2026).
- C. Schulte, M. Lagerkvist, and G. Tack. 2006. “Gecode.” *Software download and online material at the website: <http://www.gecode.org>*, 11–13.
- A. Schutt, T. Feydy, P. J. Stuckey, and M. G. Wallace. 2011. “Explaining the cumulative propagator.” *Constraints*, 16, 3, 250–282.
- A. Schutt and P. J. Stuckey. 2016. “Explaining producer/consumer constraints.” In: *International Conference on Principles and Practice of Constraint Programming*. Springer, 438–454.
- C. Thomas and P. Schaus. 2024. “A Constraint Programming Approach for Aircraft Disassembly Scheduling.” In: *International Conference on the Integration of Constraint Programming, Artificial Intelligence, and Operations Research*. Springer, 211–220.
- P. Vilim and D. O. F. Pons. 2023. *OptalCP*. OptalCP Scheduling Solver. Available online. (2023). <https://optalcp.com>.

A Proofs of Lemma and Theorems

This appendix provides the formal proofs for all lemma and theorems presented in Section 3.

A.1 Proof of Lemma 1

PROOF OF LEMMA 1. We prove the invariant maintained by the algorithm during the chronological processing of events. At any moment, the running accumulators $\underline{P}$ and $\overline{P}$ represent the contributions of all tasks whose possible time window overlaps the current time point, while $\#fp$ counts the required tasks whose fixed part overlaps the current time point.

Consider an arbitrary non-absent task $i \in O \cup R$. Its possible execution window starts at $\underline{s}_i$ and ends at $\overline{e}_i$. Therefore, the algorithm adds $\min(\underline{c}_i, 0)$ to $\underline{P}$ and $\max(\overline{c}_i, 0)$ to $\overline{P}$ at the event $\underline{s}_i$ and removes the same quantities at the event $\overline{e}_i$. As a result, the task contributes to the accumulators exactly for those time points satisfying $\underline{s}_i \leq tp.time < \overline{e}_i$.

Now suppose that i is required and has a non-empty fixed part, i.e., $\overline{s}_i < \underline{e}_i$. Its fixed part is active on the interval $[\overline{s}_i, \underline{e}_i]$. The algorithm therefore adds $\max(\underline{c}_i, 0)$ to $\underline{P}$ and $\min(\overline{c}_i, 0)$ to $\overline{P}$ when processing $\overline{s}_i$, increments $\#fp$, and performs the symmetric removal and decrement at $\underline{e}_i$. Hence a required task contributes to the fixed-part terms exactly when $\overline{s}_i \leq tp.time < \underline{e}_i$, and the counter $\#fp$ is equal to the cardinality of this set.

Because the events are processed in increasing time order, each task is inserted and removed at the correct boundary times, and no contribution is missed or counted twice. Consequently, after all events with time at most $tp.time$ have been processed, the accumulators and the fixed-part counter coincide with the sums in (2)–(3) and the cardinality in the lemma. This proves the claim. $\square$

A.2 Proof of Theorem 2

PROOF OF THEOREM 2. Suppose Algorithm 2 returns Fail at Line 15 for some time point tp . Because $tp.\#fp > 0$, at least one required task executes at time $tp.time$, so every feasible schedule must satisfy $\underline{C} \leq f(tp.time) \leq \bar{C}$, where $f(tp.time)$ denotes the resource usage. Moreover, by Lemma 1, $tp.\underline{P} \leq f(tp.time) \leq tp.\bar{P}$ holds for any schedule consistent with the current domains. Hence, any feasible schedule requires both $tp.\underline{P} \leq \bar{C}$ and $tp.\bar{P} \geq \underline{C}$. The condition at Line 15 ($tp.\underline{P} > \bar{C} \vee tp.\bar{P} < \underline{C}$) directly contradicts these requirements, proving that no feasible assignment exists and that returning Fail is sound. $\square$

A.3 Proof of Theorem 3

PROOF OF THEOREM 3. At most $4n$ events are generated (two per task for the time window, plus two for the fixed part of each required task). Sorting them requires $O(n \log n)$ time. All subsequent operations—time-point creation, accumulator updates, pointer assignments, and consistency checks—take $O(1)$ per event, contributing $O(n)$ in total. Hence the overall complexity is dominated by the sort. $\square$

A.4 Proof of Theorem 4

PROOF OF THEOREM 4. The proof follows by checking each reduction rule against the exact profile bounds delivered by Lemma 1. For any task i and any time point tp in the timeline, the values $tp.\underline{P}$ and $tp.\bar{P}$ are the minimum and maximum resource usage compatible with the current domains, while $tp.\#fp$ counts the number of required fixed parts overlapping tp .

- *Forbid*: If $tp.\underline{P} + \max(\underline{c}_i, 0) > \bar{C}$ at a time point in the forward scan, then every completion with i starting at $tp.time$ would already exceed the upper capacity. Since $tp.time < e_i \leq e_i$, any schedule with $s_i \leq tp.time$ necessarily overlaps tp ; thus i cannot start at or before $tp.time$, which leaves $s_i \geq tp.next.time$ as the only valid domain reduction. The symmetric backward argument for $\bar{e}_i$ is identical.
- *Mandatory (C3)*: Suppose tp is outside the fixed part of i ($tp.time \in [\underline{s}_i, \bar{e}_i] \setminus fp_i$) and $tp.\#fp > 0$. If

$$tp.\underline{P} - \min(\underline{c}_i, 0) > \bar{C} \quad \text{or} \quad tp.\bar{P} - \max(\bar{c}_i, 0) < \underline{C},$$

then removing the contribution of i from the profile at time tp would leave a resource load that is already incompatible with the capacity interval. Hence every feasible schedule must keep i active at tp ; otherwise the capacity constraint is violated. Furthermore, since all other tasks can collectively contribute at most $tp.\bar{P} - \max(\bar{c}_i, 0)$ at time tp , task i must supply at least the remaining deficit $\underline{C} - (tp.\bar{P} - \max(\bar{c}_i, 0))$, ensuring that tightening $\underline{c}_i \geq deficit$ is sound (and symmetrically, $\bar{c}_i \leq overload$). This is the condition encoded by the rule *checkIfMandatory*. Over the fixed part of a required task, the task is already known to execute, and the corresponding bound tightening is directly enforced by the *Consumption* rule. For an optional task, the interval $[\bar{s}_i, e_i]$ is likewise checked by *checkIfMandatory* in Step 3.

- *Consumption*: If $\bar{s}_i < e_i$, then task i executes across this interval if it is executed. If $i \in R_{init}$, this interval is its fixed part fp_i , which contributed to the profile $\mathcal{L}$ during initialization, so its full fixed-part contribution is embedded in $\mathcal{L}$ and must be removed before checking feasibility: any value $c_i > \bar{C} - (tp.\underline{P} - \underline{c}_i)$ would force the minimum profile above $\bar{C}$; similarly, any value $c_i < \underline{C} - (tp.\bar{P} - \bar{c}_i)$ would force the maximum profile below $\underline{C}$. If $i \in O_{init}$, task i did not contribute its fixed part to $\mathcal{L}$ ($fp_i = \emptyset$); only its non-fixed

contribution is removed (as in the (C3) check) to avoid counting the same task twice in the residual capacity. For tasks with $\bar{s}_i \geq \underline{e}_i$ and $\underline{d}_i \geq 1$, the task is guaranteed to execute at least at some point in $[\underline{e}_i - 1, \bar{s}_i]$, so the same argument applies to every time point in $[\underline{e}_i - 1, \bar{s}_i]$. So an assignment to c_i is only invalid if it violates capacity across every point in the interval $[\underline{e}_i - 1, \bar{s}_i]$, which is exactly what the rule enforces. If $\underline{d}_i = 0$, the task may execute with duration zero without consuming any resource, so its consumption is correctly not restricted.

- **Duration:** If a sub-interval $[a, b)$ is excluded from A , then every schedule of duration $b - a$ would force i to overlap a point where the profile cannot satisfy the capacity range. Thus no feasible assignment can keep a duration longer than the maximum feasible span recorded by the algorithm.

Therefore every domain reduction performed by Algorithm 3 removes only values that cannot belong to a solution of $\text{GeneralizedCumulative}(T, \underline{C}, \bar{C})$. $\square$

A.5 Proof of Theorem 5

PROOF OF THEOREM 5. The outer loop of Algorithm 3 (Line 3) visits every non-fixed task $i \in R_{\text{init}} \cup O_{\text{init}}$. For each task, the inner loops examine every relevant time point in the fixed timeline $\mathcal{L}$, and each call to *checkIfMandatory* is triggered on exactly the time points outside the fixed part that can witness a (C3)-conflict. We show that all conditions of Definition 7 are satisfied upon termination:

- (C1) The forward loop at Line 6 scans the timeline from $tpAt(\underline{s}_i)$ and advances $\underline{s}_i$ past all time points satisfying the (C1) condition. Upon exit, no (C1)-conflicting time point remains in $[\underline{s}_i, \min(\bar{s}_i, \underline{e}_i))$.
- (C2) The backward loop at Line 11 scans the timeline from $tpAt(\bar{e}_i).prev$ and pulls $\bar{e}_i$ before every (C2)-conflicting point. Upon exit, no (C2)-conflicting time point remains in $[\max(\bar{s}_i, \underline{e}_i), \bar{e}_i)$.
- (C3) The mandatory check is evaluated at every time point outside the fixed part visited in the forward, backward, and free-part scans (Lines 8, 13, and 31), as well as across $[\bar{s}_i, \underline{e}_i)$ for optional tasks (Line 18). Whenever a (C3)-conflict arises, the necessary presence, time-window, and consumption conditions are enforced by *checkIfMandatory*; hence no (C3)-conflicting point remains after termination.
- (C4) For tasks with $\bar{s}_i < \underline{e}_i$, the loop at Line 17 scans the interval $[\bar{s}_i, \underline{e}_i)$ and distinguishes whether the task was initially required ($i \in R_{\text{init}}$) or optional ($i \in O_{\text{init}}$) when the timeline profile was built: when $i \in R_{\text{init}}$, its full contribution is removed at Line 20; when $i \in O_{\text{init}}$, only its non-fixed contribution is removed at Line 22 to avoid double counting. The consumption domain is tightened until no (C4)-conflicting point remains.

Together, (C1)–(C4) ensure pointwise consistency (TC1). Furthermore:

- **Free-part consumption consistency (TC2):** For tasks with $\bar{s}_i \geq \underline{e}_i$ and $\underline{d}_i \geq 1$, the loop at Line 28 evaluates the minimum-overlapping interval $[\underline{e}_i - 1, \bar{s}_i]$ and updates the consumption bounds to $[\underline{c}_i, \bar{c}_i]$ at Line 38, ensuring Condition (TC2).
- **Duration consistency (TC3):** For tasks with $\bar{s}_i \geq \underline{e}_i$, the algorithm records the longest sub-interval of $[\underline{s}_i, \bar{e}_i)$ that remains free of (C1)- and (C2)-conflicts; this is exactly the feasible duration-maximizing interval used to set $\bar{d}_i$ at Line 39, satisfying Condition (TC3).

Since the timeline $\mathcal{L}$ is fixed while these scans run, the state obtained when the algorithm terminates satisfies all conditions (TC1)–(TC3) of Definition 7. Hence Algorithm 3 is timetabling filtering consistent with respect to $\mathcal{L}$. $\square$

A.6 Proof of Theorem 6

PROOF OF THEOREM 6. The timeline $\mathcal{L}$ contains $O(n)$ time points. The outer loop processes $O(n)$ tasks. For each task, each inner loop scans at most $O(n)$ time points with $O(1)$ work per time point. The total cost is therefore $O(n^2)$. $\square$